JSPM's

RAJARSHI SHAHU COLLEGE OF ENGINEERING

(An Empowered Autonomous Institute, Tathawade, Pune)

An Engineering Innovation and Society Report On

"Multimodal AI Teaching Copilot for Real-Time Visual Learning Generation"

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

(ELECTRONICS & TELECOMMUNICATION ENGINEERING)

SUBMITTED BY

1. Paritosh Vinayak Raut (RBT23ET125)
2. Prachi Vijay Potdar (RBT23ET122)

DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

A.Y. 2026-2027

JSPM's

RAJARSHI SHAHU COLLEGE OF ENGINEERING

(An Empowered Autonomous Institute)

Department of Electronics and Telecommunication Engineering

CERTIFICATE

This is to certify that the project report entitled

"Multimodal AI Teaching Copilot for Real-Time Visual Learning Generation"

Submitted by

1. Paritosh Vinayak Raut (RBT23ET125)
2. Prachi Vijay Potdar (RBT23ET122)

are Bonafide students of this institute and the work has been carried out by them under the supervision of **Mrs. Deepali Magdum** and it is approved for the fulfillment of the requirement of **Savitribai Phule Pune University,** for the award of the degree of **Bachelor of Technology** in Electronics & Telecommunication Engineering, AY-2026-27.

Mrs. Deepali Magdum
Project Guide

Mrs. D. K. Magdum
Mrs. S. P. Pattanaik
Project Coordinator

Dr.S.C.Wagaj
HOD, E&TC

Dr.A.M.Badadhe
Director, RSCOE
Pune

Place:
Date:

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be impossible without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

We have great pleasure in expressing our deep sense of gratitude to Dr. A.M.Badadhe, Director, JSPM's RSCOE, Tathawade for providing necessary infrastructure and creating a good environment.

We take this opportunity to express our profound gratitude to Dr. S.C.Wagaj, Head, Department of Electronics and Telecommunication Engineering for his constant support and encouragement. We express our gratitude to Mrs. Deepali Magdum, our project guide, for constantly monitoring the development of the project and setting up precise deadlines. Her valuable suggestions were the motivating factors in completing the work

Finally, a note of thanks to the teaching and non-teaching staff of the Department of Electronics and Telecommunication Engineering for their cooperation extended to us, those who helped us directly or indirectly during the project work.

Students Name

1. Paritosh Vinayak Raut
2. Prachi Vijay Potdar

ABSTRACT

Slides prepared before a lecture fix its content and order in advance. The examples, derivations and answers a teacher adds while explaining, which are often the parts students find hardest, never reach the screen, and drawing them on the board takes time away from teaching. Live captions show what is said word by word but give no structure, while current slide generators and large language models (LLMs) work offline from a finished document. When an LLM is simply asked to regenerate slides from a growing transcript, it rewrites what students are reading and sometimes adds content the teacher never said. The problem addressed in this project is therefore to turn a teacher's live speech into a structured, stable and faithful visual lesson on the classroom projector while the lecture is still going on.

The Multimodal AI Teaching Copilot is a software system that listens to the teacher through a laptop microphone and builds a continuity-aware slide deck on the projector during the lecture. Speech is segmented by the Silero voice activity detector and transcribed locally on the GPU by faster-whisper (Whisper medium.en, 8-bit). A deterministic filter removes classroom talk, and a concept tracker based on MiniLM sentence embeddings follows the topic. A rule-based gate groups finished sentences into bounded units, and only these units, with a short outline and summary, are sent to an LLM on a free hosted tier (gpt-oss-120b, with a Qwen3 model as the second choice). The LLM never controls the flow: it only returns a schema-validated interpretation (topic, subtopic, relation to what came before and typed content). A deterministic planner turns it into slide operations that only extend the deck, a transcript-coverage guard restores facts the LLM leaves out, a formula guard flags changed symbols, and a composer renders definition cards, key points, comparison tables, timelines, process flows, classification trees and typeset formulas, with openly licensed images checked by CLIP. A FastAPI server pushes versioned slide patches over WebSocket to the projector view, a teacher control view (editing, image search, pause, lecture structure, mistake cards) and a student view. After the lecture the event log produces study notes, an assignment, a quiz and a PPTX deck.

The second version of the system adds Concept Simulation: when the teacher says "consider a block of 5 kg on a rough floor" or "let me show you a p-n junction", a parametric simulation appears on its own slide and follows the values the teacher speaks. Twenty-one developer-written templates cover mechanics, circuits, waves, orbits, trigonometry, biological and chemical processes, RLC circuits, semiconductors, computer networks, optics and gas laws. Every number printed on the projector is either said by the teacher, set by the teacher or computed by Python solvers from such numbers; the simulation path uses no LLM tokens.

The system was evaluated on 36 held-out lectures (409 minutes) by six speakers and six public

lectures from MIT OpenCourseWare and NPTEL, all run on one laptop with a 4 GB graphics card. The median delay from the end of a spoken line to its display was 11.2 s. On 18 independently annotated lectures, slide changes matched topic boundaries with precision 0.78, 86.6 % of key facts reached a slide, only 0.9 % of statements were not supported by the speech, and five teachers rated the slides between 3.87 and 4.17 out of 5. Compared with an LLM-controlled baseline on the same units, the proposed design rewrote or removed only 2.0 % of the statements students had already seen against 38.2 %, lower in all 18 lectures. For simulations, the spoken-cue detector showed a simulation with precision and recall 1.00 over 66 recorded and scripted lectures with no false simulation on lectures that had none, and the solvers reproduce hand-computed textbook values. The results show that a bounded LLM inside a deterministic core can produce live, stable and faithful visual teaching material from ordinary speech on low-cost hardware.

Keywords: Automatic slide generation, speech recognition, large language models, educational technology, concept simulation, real-time systems, human-in-the-loop

TABLE OF CONTENTS

Sr. No.	Title of Chapter	Page No.
	CERTIFICATE	i
	ACKNOWLEDGEMENT	ii
	ABSTRACT	iii
	LIST OF ABBREVIATIONS	vii
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
01	**Chapter 1: INTRODUCTION**	1
1.1	Background of the Topic	2
1.2	Real-World Problem	3
1.3	Proposed Solution	3
1.4	Motivation	4
1.5	Scope of the Project	4
1.6	Organization of the Report	5
02	**Chapter 2: LITERATURE SURVEY**	6
2.1	Literature Survey	7
2.2	Tabular Representation of Literature Survey	10
2.3	Motivation from Literature Survey	11
03	**Chapter 3: SYSTEM DESIGN / METHODOLOGY**	12
3.1	Problem Statement	13
3.2	Objectives	13
3.3	Methodology	13
3.3.1	Block Diagram	14
3.3.2	Project Specifications	15
3.3.3	Working Principle	15
3.4	System Requirements	18
3.4.1	Hardware Requirements	18
3.4.2	Software Requirements	18
3.5	Flowchart	18
04	**Chapter 4: PROJECT IMPLEMENTATION**	20
4.1	Tools and Technology Used	21
4.2	Software Architecture and Modules	21
4.3	Speech Capture and Recognition	22
4.4	Lecture Understanding with a Bounded LLM	22
4.5	Slide Planning, Composition and Images	22
4.6	Live Display and Teacher Control View	23
4.7	Concept Simulation (Version 2)	24
4.8	Lecture Materials and Persistence	26
4.9	Testing Strategy	27
05	**Chapter 5: EXPERIMENTATION AND RESULT ANALYSIS**	28
5.1	Experimental Setup	29
5.2	End-to-End Latency	29
5.3	Topic Boundaries and Content Accuracy	31
5.4	Ablation of the Planner Rules	31
5.5	Comparison with an LLM-Controlled Baseline	32

Sr. No.	Title of Chapter	Page No.
5.6	Slide Output for Different Input Conditions	32
5.7	Concept Simulation: Outputs for Different Input Conditions	33
5.8	Simulation Detection and Value Parsing	38
5.9	Discussion of Results	39
06	**Chapter 6: CONCLUSION**	40
6.1	Advantages	41
6.2	Limitations	41
6.3	Applications	41
6.4	Conclusion	42
6.5	Future Scope	42
07	**REFERENCES**	43
08	**Appendix A: Details of Paper Publication**	45
09	**Appendix B: Implementation of Paper, Copyright Certificate, Plagiarism Report**	46
10	**SDG Mapping of Project**	47

LIST OF ABBREVIATIONS

Abbreviation	Illustration
AC1	Gwet's agreement coefficient
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
CLIP	Contrastive Language-Image Pre-training
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
ICC	Intraclass Correlation Coefficient
JSON	JavaScript Object Notation
LLM	Large Language Model
MiniLM	Miniature Language Model (distilled sentence encoder)
NLP	Natural Language Processing
NPTEL	National Programme on Technology Enhanced Learning
OCW	OpenCourseWare (MIT)
ONNX	Open Neural Network Exchange
PPTX	PowerPoint Open XML presentation
RAM	Random Access Memory
RLC	Resistor-Inductor-Capacitor circuit
RTT	Round-Trip Time
SDG	Sustainable Development Goal
STT	Speech-to-Text
SVG	Scalable Vector Graphics
TCP	Transmission Control Protocol
TIR	Total Internal Reflection
UI	User Interface
VAD	Voice Activity Detection
VRAM	Video Random Access Memory
WER	Word Error Rate

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Slide representations chosen from the type of spoken content: (a) comparison, (b) timeline, (c) classification tree, (d) cause and effect	2
3.1	Block diagram of the Multimodal AI Teaching Copilot (lanes A–C run in one Python process; only the interpreter talks to the LLM)	14
3.2	Planner decision for one interpreted unit	17
3.3	Flowchart of the live lecture pipeline	19
4.1	Slides generated live from recorded lectures in physics, mathematics, chemistry and biology	23
4.2	Teacher control view (/control) during a live chemistry lecture with the lecture structure tree	24
4.3	Simulation mode in /control: block on a surface with the computed card, the simulation bar (play, step, restart, speed, Values, Show, Full stage) and Back to slides	25
4.4	Simulations from different subjects rendered by the implemented templates	26
4.5	Materials generated from the event log of a recorded lecture: (a) study notes page, (b) assignment	27
5.1	Cumulative distributions of speech-to-text time, LLM call time and end-to-end delay	30
5.2	Timeline of a recorded chemistry lecture: LLM units, fallback units and new slides on the projector per topic	30
5.3	Block on a surface for two input conditions	34
5.4	DC circuit: series and parallel arrangement of the same resistors	35
5.5	RLC step response: underdamped and overdamped cases with VC(t) and I(t) graphs	35
5.6	Thin lens: real image and virtual image with the three principal rays	36
5.7	Refraction: air to glass and total internal reflection from glass to air	36
5.8	p-n junction under forward and reverse bias	37
5.9	Water cycle: first step and all steps reached by speech	38
5.10	Shortest-path routing (Dijkstra): an intermediate step and the final path	38

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature summary	10
2.2	Comparison of the proposed system with prior systems	11
3.1	Project specifications	15
3.2	Hardware requirements	18
3.3	Software requirements	18
4.1	Tools and technologies used	21
4.2	Simulation templates implemented	24
5.1	Evaluation data	29
5.2	Live system performance (seconds, from session logs)	29
5.3	Accuracy and teacher ratings (18 annotated D2 lectures)	31
5.4	Planner ablation by replay (18 annotated D2 lectures)	32
5.5	Proposed system vs. LLM-controlled baseline (same D2 units)	32
5.6	Spoken input and the resulting slide representation	33
5.7	Block on a surface: push F = 20 N for different mass and friction	33
5.8	Block on an incline: m = 5 kg for different angles and friction	34
5.9	Simple pendulum (amplitude 10°) for different lengths	34
5.10	Projectile launched from the ground for different speeds and angles	34
5.11	DC circuit and series RLC step response for different inputs	35
5.12	Series RLC with AC supply (V = 10 V, R = 100 Ω, L = 0.1 H, C = 10 μF) at different frequencies	35
5.13	Thin lens (f = 15 cm) for different object distances, Cartesian sign convention	36
5.14	Refraction at a plane boundary (Snell's law)	36
5.15	Further templates with different inputs	37
5.16	Spoken sentences and the simulation system's response	38

CHAPTER 1: INTRODUCTION

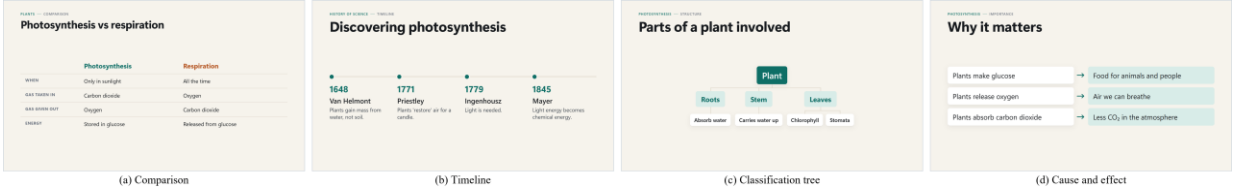
1.1 Background of the Topic

Classroom teaching is still built around a teacher who explains a topic aloud and a visual medium that supports the explanation: the blackboard, the overhead projector and, for the last two decades, the slide deck. Each medium has a weakness. A board drawing follows the teacher's explanation exactly, but it is slow to draw, hard to read from the back of a large room and erased before students have copied it. A slide deck is clean and readable, but it is prepared before the lecture, so it fixes the content and the order of the lesson in advance.

Research on multimedia learning explains why the visual channel matters. In Mayer's experiments, learners who received words together with matching pictures remembered more and solved transfer problems better than learners who received words alone [1]. Cognitive load theory by Sweller explains the limit behind this effect: a learner can hold only a few new elements in working memory at once, so material that is long, verbatim or decorative adds extraneous load and reduces learning [2], [3]. Good teaching visuals are therefore short, structured and closely tied to what is being said at that moment.

Three technologies have recently made it possible to produce such visuals automatically. First, automatic speech recognition (ASR) has become accurate and robust. Whisper, trained on 680,000 hours of weakly supervised audio, transcribes lecture speech with a low word error rate without being tuned for a particular speaker [4], and optimized implementations such as faster-whisper run it on an ordinary laptop GPU. Second, sentence embedding models such as MiniLM represent the meaning of a sentence as a vector, so that a change of topic can be measured as a drop in similarity [5]. Third, large language models (LLMs) can read a piece of text and return a structured interpretation of it: what the topic is, which statements are definitions, which are steps of a process and which are formulas [6], [7].

Combining these technologies gives the idea behind this project: a *teaching copilot* that listens to the teacher, understands the topic, the continuity and the meaning of the explanation, and renders a designed live educational display on the classroom projector while the teacher is still speaking. The teacher teaches normally; the copilot acts as an assistant who prepares the board. Fig. 1.1 shows the kind of representations the system chooses automatically depending on what is said.



(a) Comparison (b) Timeline (c) Classification tree (d) Cause and effect

Figure 1.1: Slide representations chosen from the type of spoken content: (a) comparison, (b) timeline, (c) classification tree, (d) cause and effect

The project was built in versions on a common core. Version 1 (V1) produces live slides from speech. Version 2 (V2), *Concept Simulation*, adds live parametric simulations: when the teacher sets up a

physical or technical situation in words, a moving, correctly computed drawing appears and follows the numbers the teacher says. Later versions (story scenes and multilingual output) are planned to reuse the same core.

1.2 Real-World Problem

In an ordinary engineering or school classroom the following problems are observed every day:

- **Prepared slides do not follow the lecture.** The examples, derivations, side questions and answers that a teacher adds while explaining never reach the screen. Students either copy them from speech, which they cannot do fast enough, or lose them.
- **Board work costs teaching time.** Drawing a free-body diagram, a circuit or a ray diagram by hand takes minutes, and the drawing is static: the teacher cannot show what happens when the mass doubles or the resistance changes without drawing it again.
- **Live captions are not a lesson.** Real-time captions help students with hearing difficulties, but verbatim text is hard to follow and gives no structure; Kushalnagar et al. found that students struggle to read and understand real-time classroom captions [8].
- **LLM output is unstable and not always faithful.** When an LLM is asked to regenerate a slide from a growing transcript, it rephrases and reorders what students are reading, removes points they were copying and sometimes adds facts the teacher never said, a problem known as hallucination [9].
- **Cost and infrastructure.** Many institutions cannot pay for commercial AI services or powerful servers. A useful system must run on one laptop, use free tiers and local models, and keep working when the network or the AI service fails.

These problems can be summarized as three gaps. **P1:** what is projected does not follow what is being taught. **P2:** live speech is never turned into a structured visual lesson. **P3:** LLM output is neither stable nor faithful to the teacher's words. In addition, for science and engineering topics, **P4:** a static picture cannot show how a system responds when its parameters change.

1.3 Proposed Solution

The Multimodal AI Teaching Copilot closes these gaps with a design in which the LLM is a gated reasoning service inside a deterministic core. The core decides when to call the LLM, what bounded context to send, and what to do with the answer. The LLM only labels a unit of speech; it never decides when a slide is created, replaced or deleted. The main ideas are:

1. Incremental slide generation from live speech, with local speech recognition and a concept tracker that follows the topic from sentence embeddings.

2. An extend-only update policy: a deterministic planner executes only operations that add to the deck, so nothing a student has read is rewritten by the model.
3. Transcript-grounding safeguards: a coverage guard that puts back facts the LLM omitted, a formula guard, and mistake cards that let the teacher decide on suspected errors.
4. Teacher control: a control view in which the teacher can edit, pause, blank the screen, add images and override every automatic decision.
5. Concept Simulation (V2): spoken setups and values drive parametric simulations whose numbers come only from the teacher or from Python solvers.
6. Post-lecture materials: study notes, assignments, quizzes and a PPTX deck generated from the recorded event log of the taught lecture.

1.4 Motivation

The motivation of the project comes from the authors' own experience as students. In many lectures the most useful explanation is given orally, between two slides, and is not available afterwards. A system that captures this explanation as structured slides and simulations would help students who take notes slowly, students who join late or miss a lecture, students with hearing difficulties and students studying in a second language. For teachers, it removes the need to prepare a complete deck for every topic and lets them adapt the lesson to the class without losing the visual support.

From an engineering point of view, the project is a real-time system that combines signal processing (audio capture and voice activity detection), machine learning (speech recognition, embeddings, image ranking), natural language understanding with an LLM, deterministic control logic, numerical simulation and a real-time web display. Building it with free and local components on a 4 GB GPU laptop makes it affordable for Indian institutions.

1.5 Scope of the Project

- Input: the teacher's English speech from a laptop microphone (or a recorded audio file / transcript for testing).
- Output: a live slide deck on the projector (/display), a teacher control view (/control), a read-only student view (/view), and post-lecture materials.
- Subjects: general school and engineering subjects; slides are subject-independent, simulations cover 21 templates from mechanics to computer networks.
- Hardware: one laptop with an NVIDIA GPU of 4 GB; LLM on a free hosted tier; no paid API.
- Out of scope: languages other than English, generated video, and grading of student learning outcomes.

1.6 Organization of the Report

Chapter 2 reviews published work on slide generation, speech recognition, LLMs in education and hallucination, and summarizes it in a table. Chapter 3 gives the problem statement, objectives, block diagram, specifications, working principle, requirements and flowchart. Chapter 4 describes the implementation of each module, the user interfaces and the Concept Simulation version. Chapter 5 presents the experiments and analyses the results for different inputs. Chapter 6 lists advantages, limitations, applications, conclusions and future scope, followed by references and appendices.

CHAPTER 2: LITERATURE SURVEY

2.1 Literature Survey

The survey covers papers published between 2020 and 2025 in four areas related to the project: automatic slide and presentation generation, speech and vision models used as components, LLMs in education and their reliability, and structured generation. For each paper the method used and the results achieved are summarized, followed by its relevance to this project.

2.1.1 DOC2PPT: Automatic Presentation Slides Generation from Scientific Documents (Fu et al., AAAI 2022)

Fu et al. introduced the task of generating a slide deck directly from a scientific paper [10]. Their hierarchical sequence-to-sequence model reads the document section by section, uses a progress tracker to decide when to move to the next slide, places figures with an object placer and shortens sentences with a paraphrasing module. They built a dataset of paper–slide pairs for training and evaluation. The model produced decks that covered the paper better than extractive baselines in automatic and human evaluation. The work treats slide creation as an offline process over a complete, clean document.

2.1.2 D2S: Document-to-Slide Generation via Query-Based Text Summarization (Sun et al., NAACL 2021)

Sun et al. split slide generation into two steps [11]. Slide titles are used as queries to retrieve the relevant passages and figures of a paper, and a long-form question-answering model then writes the bullet text for each title. The method was trained and evaluated on a new dataset of paper and slide pairs, and it outperformed summarization baselines on ROUGE scores and human ratings. The paper shows that slide content is best generated per slide, guided by a heading, an idea reused here where content is attached to the current topic and subtopic.

2.1.3 Grounded LLMs for Educational Presentation Generation (Xie et al., 2025)

Xie et al. used LLMs to generate educational presentations and grounded the generation in retrieved source material so that the slide content can be traced to references [12]. Their pipeline plans an outline, retrieves supporting sources and writes slides from them. Grounding reduced unsupported content compared with an LLM writing from its own knowledge. Like the other generators, it builds the whole deck at once from a topic or document; the teacher's live words are not its source.

2.1.4 PPTAgent: Generating and Evaluating Presentations beyond Text-to-Slides (Zheng et al., EMNLP 2025)

Zheng et al. proposed an agent that creates presentations by editing reference slides instead of writing them from scratch [13]. In a first stage it analyses reference decks to learn slide types and content schemas; in a second stage it drafts an outline and applies editing actions to selected reference slides. They also proposed PPTEval, which uses an LLM as a judge on content, design and coherence.

PPTAgent scored higher than earlier generation methods on all three dimensions. The paper shows the value of fixed layouts, which this project uses as typed slide templates.

2.1.5 AutoPresent: Designing Structured Visuals from Scratch (Ge et al., CVPR 2025)

Ge et al. created SlidesBench, a benchmark of slide-generation tasks from natural-language instructions, and trained a model that writes slides as programs which are then executed [14]. Generating slides as code gave better layout control than generating images directly, and their trained open model approached the quality of strong closed models. The paper supports a structured representation of slides; in this project the LLM is not allowed to write code, but layouts are produced by deterministic code from typed content.

2.1.6 Robust Speech Recognition via Large-Scale Weak Supervision (Radford et al., ICML 2023)

Radford et al. trained Whisper, an encoder–decoder transformer, on 680,000 hours of multilingual audio with weak transcripts collected from the internet [4]. Without fine-tuning, Whisper generalizes across datasets, accents and noise, and its robustness approaches that of human transcribers. The model is released in several sizes. The project uses Whisper medium.en through faster-whisper with 8-bit weights, which fits beside other models on a 4 GB GPU and gave a word error rate of 7.1 % on a lecture with human captions.

2.1.7 Learning Transferable Visual Models from Natural Language Supervision (Radford et al., ICML 2021)

CLIP was trained contrastively on about 400 million image–text pairs so that an image and its description have similar embeddings [15]. This allows zero-shot classification: any image can be scored against any text prompt. CLIP matched supervised models on many benchmarks without task-specific training. In the project, CLIP (ONNX, CPU) checks that a retrieved openly licensed image really shows the object being taught and is not a logo, a map or a diagram of something else.

2.1.8 MiniLM: Deep Self-Attention Distillation (Wang et al., NeurIPS 2020)

Wang et al. compressed large transformer models by distilling the self-attention distributions and value relations of the teacher's last layer into a small student model [5]. The student kept most of the teacher's accuracy on language understanding benchmarks with a fraction of the parameters and computation. A MiniLM sentence encoder is used in this project to embed each utterance on the CPU in a few milliseconds and to measure topic shifts by cosine distance.

2.1.9 Automatically Generated Summaries of Video Lectures (Gonzalez et al., BEA 2023)

Gonzalez et al. used an LLM to generate summaries from the transcripts of recorded video lectures and studied their effect on students [16]. Students who received the summaries reported a better experience,

and their quiz results indicated that summaries may enhance learning. The work confirms that structured text from lecture speech is useful, but it is produced after the lecture and as plain text, not as a live visual lesson.

2.1.10 ChatGPT for Good? Opportunities and Challenges of LLMs for Education (Kasneci et al., 2023)

Kasneci et al. reviewed the opportunities of LLMs for students and teachers, such as personalized explanations, generation of exercises and support for writing, together with the risks: wrong or biased output, over-reliance, privacy and copyright [6]. They argue that teachers must stay in charge and need competencies to judge model output. This project follows that recommendation: every automatic decision can be overridden by the teacher, and doubtful statements are shown to the teacher instead of the class.

2.1.11 Survey of Hallucination in Natural Language Generation (Ji et al., ACM CSUR 2023)

Ji et al. surveyed hallucination, i.e. generated text that is unfaithful to the source or not supported by it, across summarization, dialogue, question answering, translation and other tasks [9]. They distinguish intrinsic hallucination (contradicting the source) from extrinsic hallucination (adding unverifiable content) and review metrics and mitigation methods. The survey motivated the transcript-coverage guard and the measurement of unsupported statements in this project.

2.1.12 A Systematic Review of Automatic Question Generation for Education (Kurdi et al., IJAIED 2020)

Kurdi et al. reviewed automatic question generation systems for educational purposes, covering their inputs, generation methods, question types and evaluation [17]. Most systems generated factual questions from text, and evaluation methods were not standardized, which made comparison difficult. The review guided the design of the quiz and test generator of the post-lecture materials, which only uses taught content and is edited by the teacher before use.

2.1.13 Efficient Guided Generation for Large Language Models (Willard and Louf, 2023)

Willard and Louf reformulated constrained text generation as transitions of a finite-state machine over the model's vocabulary, so that the output is guaranteed to match a regular expression or a grammar such as a JSON schema with little overhead per token [7]. This fixes the format of the output but not its facts. In this project the LLM output is validated against a JSON schema with Pydantic and repaired once, and the content is separately checked against the transcript.

2.2 Tabular Representation of Literature Survey

Table 2.1: Literature summary

Sr. No.	Title of Paper	Author	Publication & Year	Methodology Used	Observations
1	DOC2PPT: Automatic presentation slides generation from scientific documents	T.-J. Fu et al.	AAAI, 2022	Hierarchical seq2seq with progress tracker, object placer, paraphraser	Offline; whole deck from a clean paper
2	D2S: Document-to-slide generation via query-based text summarization	E. Sun et al.	NAACL-HLT, 2021	Title-based retrieval + long-form QA summarization	Per-slide generation guided by headings
3	Leveraging grounded LLMs to automate educational presentation generation	E. Xie et al.	PMLR vol. 264, 2025	LLM outline + retrieval-grounded slide writing	Grounding reduces unsupported content
4	PPTAgent: Generating and evaluating presentations beyond text-to-slides	H. Zheng et al.	EMNLP, 2025	Edit-based agent over reference slides; PPTEval	Better content, design, coherence; offline
5	AutoPresent: Designing structured visuals from scratch	J. Ge et al.	IEEE/CVF CVPR, 2025	Slides generated as executable programs; SlidesBench	Code gives layout control
6	Robust speech recognition via large-scale weak supervision	A. Radford et al.	ICML, 2023	Transformer ASR trained on 680k h weakly supervised audio	Robust zero-shot transcription
7	Learning transferable visual models from natural language supervision	A. Radford et al.	ICML, 2021	Contrastive image–text pre-training (CLIP)	Zero-shot image scoring against text
8	MiniLM: Deep self-attention distillation	W. Wang et al.	NeurIPS, 2020	Distillation of last-layer self-attention	Small, fast sentence encoder
9	Automatically generated summaries of video lectures may enhance students' learning experience	H. Gonzalez et al.	BEA Workshop, 2023	LLM summaries of lecture transcripts, user study	Useful, but offline plain text
10	ChatGPT for good? Opportunities and challenges of LLMs for education	E. Kasneci et al.	Learn. Individ. Differ., 2023	Review / position paper	Teachers must stay in control
11	Survey of hallucination in natural language generation	Z. Ji et al.	ACM Comput. Surv., 2023	Survey of definitions, metrics, mitigation	LLM output can be unfaithful to source
12	A systematic review of automatic question generation for educational purposes	G. Kurdi et al.	Int. J. AIED, 2020	Systematic literature review	Mostly factual questions; weak evaluation

Sr. No.	Title of Paper	Author	Publication & Year	Methodology Used	Observations
13	Efficient guided generation for large language models	B. T. Willard, R. Louf	arXiv, 2023	FSM-constrained decoding (regex, JSON schema)	Fixes format, not facts

2.3 Motivation from Literature Survey

The survey shows clear progress in each component but no system that joins them for a live classroom. Slide generators [10]–[14] produce good decks, but all of them work offline from clean written input and build the whole deck at once. Lecture summaries [16] are produced after the lecture and are plain text. Captions from ASR [4], [8] are live but verbatim and unstructured. LLMs are powerful interpreters [6], but their output may be unfaithful [9], and constraining the format [7] does not make the facts correct.

From this the following design decisions were derived for the project:

- Take continuous live speech as input and update the display at the pace of the lecture (gap P1, P2).
- Use the LLM only as a bounded interpreter of small units, with schema validation; keep the slide sequence under a deterministic planner that only extends the deck (gap P3).
- Check every answer against the transcript and show suspected mistakes to the teacher, not to the class [6], [9].
- Use typed layouts and vetted images to keep visuals short and relevant, as multimedia learning research recommends [1]–[3].
- Extend the same principle to simulations: the numbers come from the teacher and from solvers, never from the model (gap P4).

Table 2.2: Comparison of the proposed system with prior systems

System	Input	Produced	Topic structure	Faithfulness to source	Slide sequence
Live captions [4], [8]	Speech	Live	None (text stream)	Verbatim text	None
Lecture summaries [16]	Recorded lecture	Offline	None (summary)	LLM summary	None
DOC2PPT, D2S [10], [11]	Paper / document	Offline	Document sections	Extracted from document	Whole deck at once
LLM slide generators [12]–[14]	Topic, document	Offline	LLM outline / plan	Retrieved sources or none	Whole deck at once
Proposed system	Live speech	During lecture	Online topic / subtopic	Coverage + formula guards	Extend-only + teacher

CHAPTER 3: SYSTEM DESIGN / METHODOLOGY

3.1 Problem Statement

To design and implement a low-cost software system that listens to a teacher's live lecture, understands its topic, continuity and meaning, and generates on the classroom projector, during the lecture, a structured, stable and transcript-faithful visual lesson consisting of designed slides, typeset formulas, relevant images and parametric concept simulations, while keeping the teacher in control and running on a single laptop with free and local AI components.

3.2 Objectives

1. To capture and transcribe classroom speech locally in real time with voice activity detection and a GPU speech-to-text model.
2. To track the lecture topic and detect topic boundaries online from sentence embeddings and spoken cues.
3. To interpret bounded units of speech with an LLM that returns only schema-validated labels and content, under a fixed token budget on a free tier.
4. To plan slide operations deterministically so that the deck only grows and nothing students have read is rewritten by the model.
5. To keep the slides faithful to the speech with transcript-coverage and formula guards and teacher-reviewed mistake cards.
6. To render typed slide layouts, formulas and vetted images on a projector view, with a teacher control view and a student view.
7. To show parametric concept simulations driven by spoken setups and values, with every printed number said, set or computed.
8. To evaluate latency, topic-boundary accuracy, content accuracy, stability and teacher ratings on held-out lectures and against an LLM-controlled baseline.

3.3 Methodology

The system is organized as a pipeline of four lanes that share one Python process and communicate through typed events. Every event is stored in an SQLite event log, which is used for recovery, replay, evaluation and post-lecture materials. Four rules hold throughout: (1) the LLM produces only labels and performs no slide operation; (2) it receives only bounded parts of the transcript, never the whole lecture; (3) a new concept is not automatically a new slide; (4) user interfaces never change the state directly, they send commands to the core.

3.3.1 Block Diagram

Fig. 3.1 shows the block diagram of the implemented system. Lane A captures and transcribes speech on the GPU. Lane B understands the lecture: it filters the transcript, tracks the concept, buffers content until the gate fires and calls the LLM through a router. Lane C is the deterministic presentation core with the lecture state, planner, composer, deck and image service. Lane D contains the browser clients.

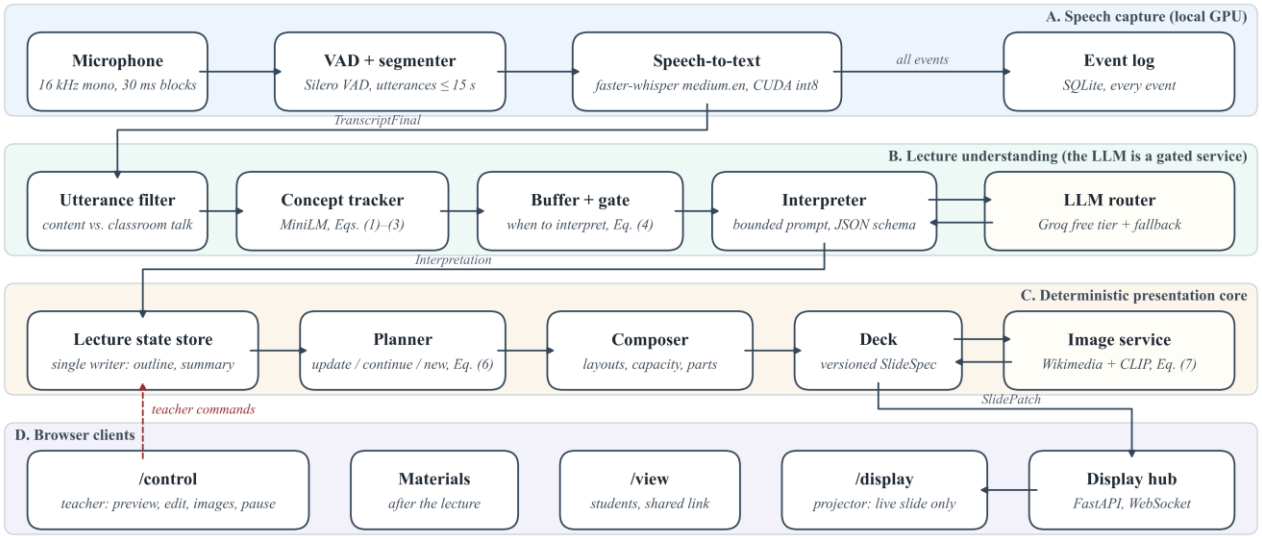


Figure 3.1: Block diagram of the Multimodal AI Teaching Copilot (lanes A–C run in one Python process; only the interpreter talks to the LLM)

3.3.2 Project Specifications

Table 3.1: Project specifications

Parameter	Specification
Input	Teacher's English speech, laptop or USB microphone, 16 kHz mono, 30 ms blocks
Speech segmentation	Silero VAD, utterances up to 15 s
Speech-to-text	faster-whisper, Whisper medium.en, CUDA int8, beam size 1, keyword prompt
Topic tracking	MiniLM sentence embeddings (ONNX, CPU), cosine shift score, threshold τ = 0.75
LLM	Groq free tier: openai/gpt-oss-120b, then qwen/qwen3 model; deterministic fallback
Prompt budget	< 3,000 tokens per call; outline ≤ 150 tokens; summary ≤ 120 tokens; temperature 0.2
Call rate	Gate-controlled, at least 8 s between calls
Slide operations	update, continue, new, retitle, noop (extend-only)
Slide layouts	Definition, key points, fact tiles, process flow, comparison, timeline, cause-effect, classification tree, formula, image, simulation
Images	Wikimedia / Openverse open licences, CLIP ViT-B/32 (ONNX int8) check
Simulations	21 parametric templates, SVG at 1100 × 740 viewBox, Python solvers
Output views	/display (projector), /control (teacher), /view (students), FastAPI + WebSocket
Median latency	11.2 s from end of spoken line to display (36 lectures)
Hardware	Laptop, NVIDIA RTX 3050 4 GB, 16 GB RAM

3.3.3 Working Principle

Speech capture. Audio from the microphone is read in 30 ms blocks at 16 kHz. Silero VAD marks speech and silence and cuts utterances of at most 15 s. Each utterance is transcribed by faster-whisper on the GPU, with a keyword prompt that contains the subject's terms, and published as a *TranscriptFinal* event.

Utterance filter and concept tracker. A deterministic filter separates content from classroom management ("take out your books"), meta talk and fillers. Each content utterance is embedded with MiniLM. With the L2-normalized embedding e_t and the concept centroid c_{t-1}, the shift score is the cosine distance used with Sentence-BERT embeddings [21]:

$$s_t = 1 - \mathbf{e}_t \cdot \mathbf{c}_{t-1} \tag{1}$$

and the centroid is an exponentially weighted mean of the embeddings, with $\alpha = 0.35$, raised to 0.6 at a boundary so that the centroid re-seeds on the new concept:

$$\mathbf{c}_t = \mathrm{norm}((1-\alpha)\mathbf{c}_{t-1} + \alpha\mathbf{e}_t) \tag{2}$$

Following the similarity-drop idea of TextTiling [20], a boundary is signalled when a cue phrase κ_t ("moving on to", "next") is heard or the shift score crosses the threshold:

$$b_t = \kappa_t \vee (s_t \geq \tau), \tau = 0.75 \quad (3)$$

The threshold is high because shift scores within one facet of a topic were observed between 0.3 and 0.6.

Gate and token budget. Content accumulates in a buffer until a rule-based gate fires. With p_t the silence after a finished sentence (s), w_t the number of buffered words and a_t the age of the oldest pending line (s):

$$G_t = (p_t \geq 1.2 \wedge w_t \geq 6) \vee w_t \geq 40 \vee b_t \vee a_t \geq 12 \quad (4)$$

with a floor of 8 s between calls, chosen from the free quota of 200,000 tokens per key and model per day.

Bounded interpretation. Each prompt carries the subject and level, an outline of at most 150 tokens, the current slide with numbered items, a rolling summary of at most 120 tokens and the new lines. The answer is JSON: topic and subtopic, the relation of the unit to what came before (same concept, elaboration, sub-concept, sibling, new topic or digression), typed content, an image hint and *concerns*, statements the model suspects are wrong. Pydantic validates the answer; if one repair also fails, a deterministic fallback shows the spoken sentences as points.

Coverage guard. Because the model sometimes drops a fact, every content sentence u of six or more words is compared with the output O. With $W(\cdot)$ the set of word stems, a recall-style overlap as in ROUGE-1 [22] decides:

$$|W(u) \cap W(O)| \leq \frac{1}{3}|W(u)| \Rightarrow u \text{ is shown as said} \quad (5)$$

A formula guard flags any token of a spoken formula that the model changed. A correction is applied only when its confidence is at least 0.75 for a factual error or 0.4 for a mis-hearing; otherwise the slide keeps what was said and the teacher receives a card with both versions.

Planner. For each interpretation the planner chooses one operation: *update* the working slide, *continue* on a new slide of the same topic, open a *new* topic slide, *retitle*, or do nothing. One unit may be misread, so a topic change must be confirmed:

$$C_n = B_n \vee \left(\bar{s}_n^{\max} \geq \tau\right) \vee \left(T_n \equiv \tilde{T}_{n-1}\right) \quad (6)$$

where B_n is true if a boundary (3) occurred in unit n, $\bar{s}_n^{\max}$ is its largest shift score smoothed over two utterances, T_n its topic and $\tilde{T}_{n-1}$ an unconfirmed candidate from the previous unit. A minimum dwell time of 15 s (6 s for the next part of a full slide) prevents rapid changes, near-duplicates (similarity ≥ 0.85) are merged and teacher edits are never overwritten. Fig. 3.2 shows the planner decision.

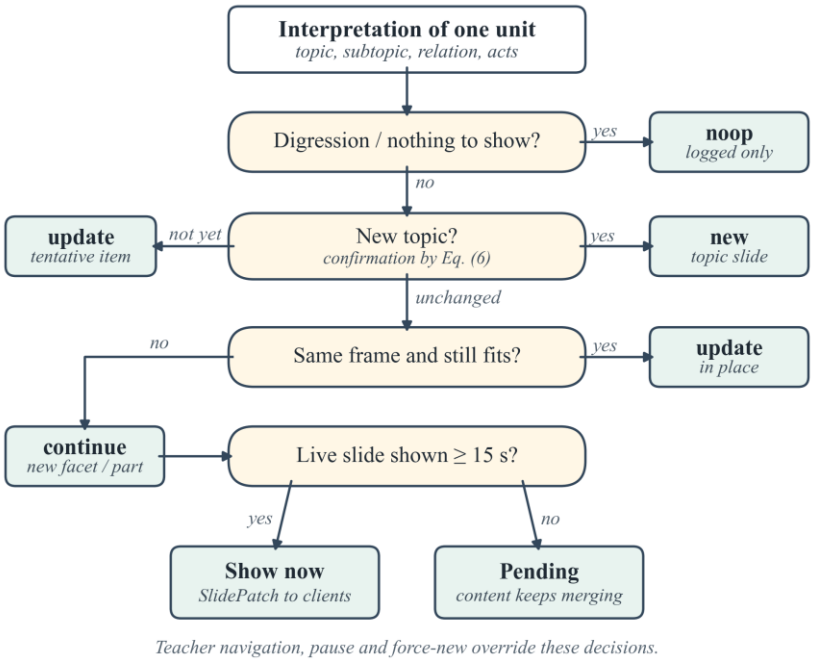


Figure 3.2: Planner decision for one interpreted unit

Composer and images. Items become typed layouts; a full slide continues in a further part (I, II, …) instead of shrinking the text, and spoken mathematics is typeset with KaTeX. For a concrete object the LLM may suggest an image query. An openly licensed image is placed only if CLIP scores it against the query above a floor and above generic distractor prompts d ("a logo", "a map"):

$$\sigma(q, I) \geq 0.22 \wedge \sigma(q, I) - max_d \sigma(d, I) \geq 0.01 \quad (7)$$

Concept Simulation. A spoken cue ("consider", "imagine", "let me show you") with the setup of a template ("a block on a rough floor", "a series RLC circuit") opens a simulation on its own slide. A deterministic value parser reads numbers with units from later sentences ("the coefficient of friction is zero point two"), checks each against the slot's range, and the template's Python solver computes the derived values. While a simulation runs no new slides are made; when it ends, a recap slide collects the points said meanwhile.

3.4 System Requirements

3.4.1 Hardware Requirements

Table 3.2: Hardware requirements

Component	Minimum / used configuration
Processor	Intel Core i5 / AMD Ryzen 5 class, 4 cores or more
GPU	NVIDIA GPU with CUDA and 4 GB VRAM (RTX 3050 Laptop GPU used)
Memory	16 GB RAM
Storage	10 GB free (models, event logs, images)
Microphone	Laptop built-in or USB / lapel microphone
Display	Projector or second screen (HDMI); any device with a browser for students
Network	Internet for the LLM free tier and image search (system degrades gracefully without it)

3.4.2 Software Requirements

Table 3.3: Software requirements

Software	Version / purpose
Operating system	Windows 10/11 (64-bit)
Python	3.10 with asyncio; virtual environment
Speech	faster-whisper ≥ 1.2 (CTranslate2), Silero VAD, sounddevice, NVIDIA CUDA 12 runtime
Understanding	onnxruntime, tokenizers, MiniLM; httpx client for the Groq API
Validation	Pydantic 2
Server / UI	FastAPI, Uvicorn, WebSocket; HTML, CSS, JavaScript; KaTeX; SVG
Images	CLIP ViT-B/32 (ONNX int8), Pillow
Materials	Playwright (headless Edge for PDFs), python-pptx, segno (QR codes)
Storage	SQLite event log
Testing	pytest, pytest-asyncio, Playwright browser tests
Browser	Microsoft Edge / Google Chrome

3.5 Flowchart

Fig. 3.3 shows the flowchart of one lecture: the loop from audio to the display repeats for every utterance until the teacher ends the lecture, after which the event log is used for the materials.

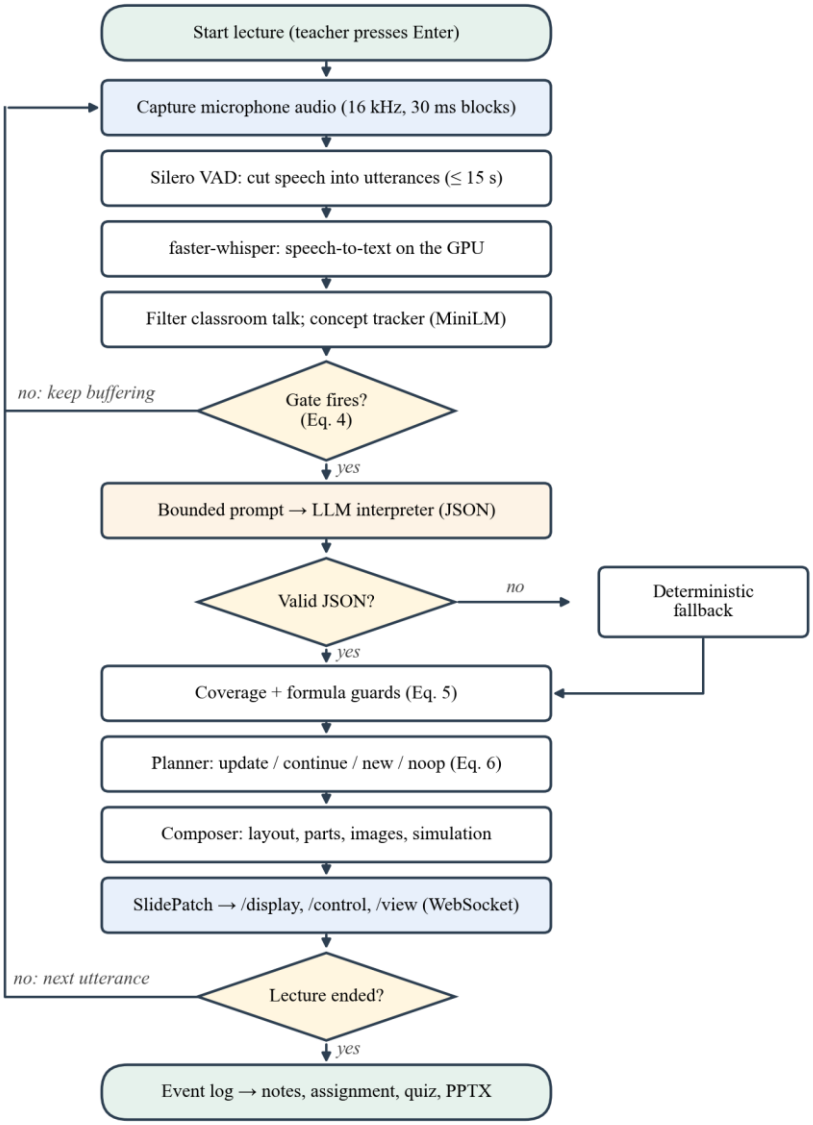


Figure 3.3: Flowchart of the live lecture pipeline

CHAPTER 4: PROJECT IMPLEMENTATION

4.1 Tools and Technology Used

The system is written in Python (about 27,400 lines in the core package) and JavaScript (about 6,400 lines in the browser views), with 86 test files. All components run on one laptop. The tools and their roles are listed in Table 4.1.

Table 4.1: Tools and technologies used

Tool / technology	Used for
Python 3.10, asyncio	Single-process event-driven core; typed events between lanes
faster-whisper (Whisper medium.en)	Local speech-to-text on the GPU, 8-bit weights
Silero VAD	Voice activity detection, utterance segmentation
MiniLM (ONNX Runtime)	Sentence embeddings for concept tracking and near-duplicate detection
Groq API (gpt-oss-120b, Qwen3)	Bounded interpretation of speech units (free tier)
Pydantic	Schema validation of LLM answers, slide and simulation contracts
SQLite	Event log for recovery, replay, evaluation and materials
FastAPI, WebSocket	Server and versioned slide patches to all views
HTML, CSS, JavaScript, KaTeX, SVG	Projector, control and student views; formulas; simulations
CLIP ViT-B/32 (ONNX)	Checking retrieved images against the taught object
Wikimedia Commons, Openverse	Openly licensed images with source, author and licence
Playwright, python-pptx	Browser tests; PDF notes and PPTX export
pytest	Unit, integration and browser tests

4.2 Software Architecture and Modules

The source code is divided into packages that follow the lanes of the block diagram. Each package exchanges typed events through an in-process event bus; only the lecture state store writes the lecture state.

- **audio, stt:** microphone capture, VAD segmentation and faster-whisper transcription with GPU warm-up; a microphone check tool lists devices and records a test.
- **understanding:** utterance filter, concept tracker, buffer and gate, prompt builder, interpreter, coverage and formula guards, glossary of lecture terms.
- **llm:** a router over several free-tier keys and two models with quota tracking; on failure the deterministic fallback is used.
- **core:** lecture state (outline, summary, topics), event definitions and the command handler.
- **presentation:** planner, composer, layouts, equation library, recap slides and the simulation controller.
- **visuals:** image search, licence capture and CLIP re-ranking; figures from the teacher's own PDF notes.

- **simulation:** template registry, 21 templates with Python solvers, spoken-cue detector, value parser and policy (Auto / Suggest / Off).
- **display, app:** FastAPI server, WebSocket hub, /display, /control and /view.
- **persistence, postlecture, materials, assess:** event log, recovery after a crash, past-lecture archive, notes, assignments, PPTX, quizzes and tests.

4.3 Speech Capture and Recognition

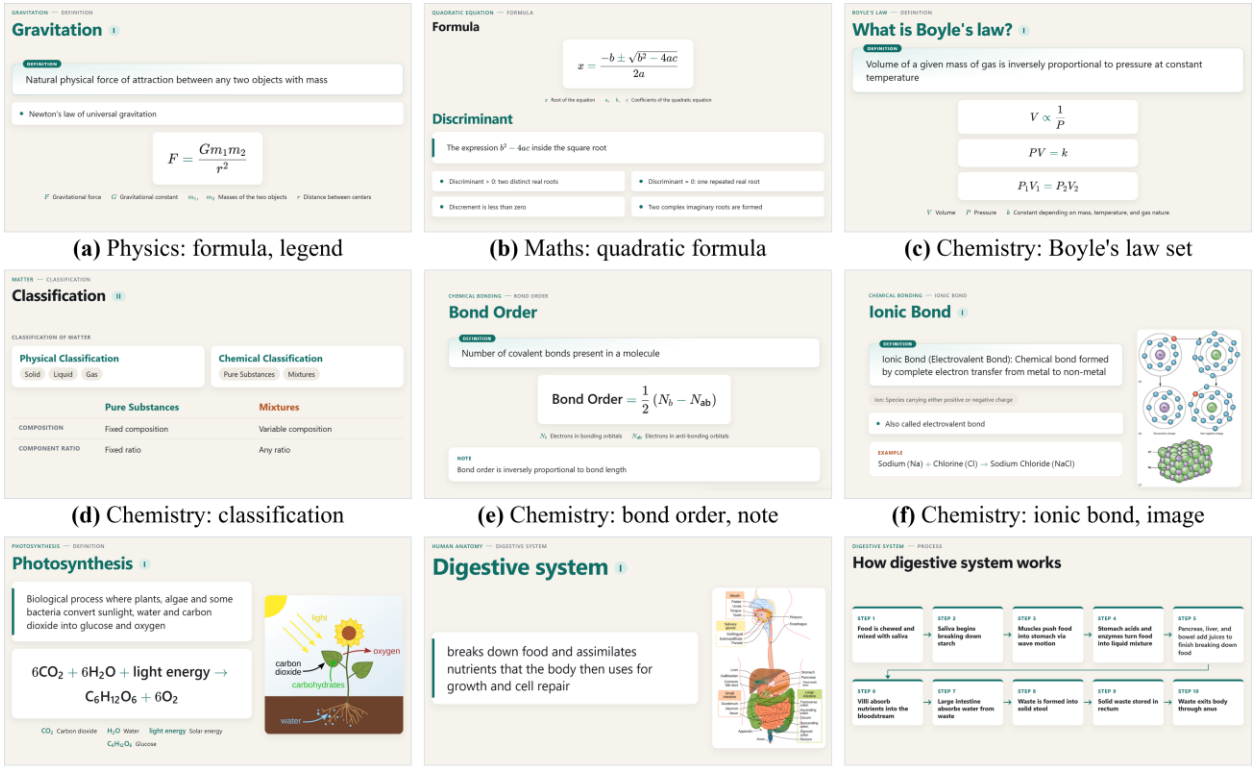
The audio module reads the microphone with sounddevice in 30 ms blocks. Silero VAD [25], which ships inside faster-whisper as an ONNX model, labels each block; an utterance ends after a pause or after 15 s. The speech-to-text module (faster-whisper [24]) loads Whisper medium.en [4] with CUDA int8 compute, uses beam size 1 for speed and passes a keyword prompt built from the lecture subject so that technical terms are spelled correctly. The median transcription time is 0.68 s per utterance. Against MIT's human captions of a public lecture (1,257 words) the live transcript had a word error rate of 7.1 %.

4.4 Lecture Understanding with a Bounded LLM

The interpreter is the only module that talks to the LLM. It builds a prompt of under 3,000 tokens from the bounded context described in Section 3.3.3, sends it to the router and validates the JSON answer. The router rotates between free keys and switches from gpt-oss-120b [18] to a Qwen3 model [19] when the daily quota is spent. Of 1,619 held-out units, 71.6 % were interpreted by the first model, 24.0 % by the second and 4.4 % by the deterministic fallback. Because the LLM only labels the unit, the lecture continues even if the service fails; the fallback simply shows what was said.

4.5 Slide Planning, Composition and Images

The planner implements the decision of Fig. 3.2 and the confirmation rule (6). The composer converts typed content into layouts; it measures the capacity of a slide and continues in a numbered part instead of shrinking text. Spoken formulas ("F equals G m1 m2 by r squared") are converted deterministically to LaTeX and rendered with KaTeX, with a legend of the symbols. A library of named equations recognizes laws such as Boyle's law or Kirchhoff's laws. Images are searched only for concrete objects, accepted by CLIP according to (7) and shown with source and licence. The teacher can remove, replace, upload or drag in an image.



(a) Physics: formula, legend **(b)** Maths: quadratic formula **(c)** Chemistry: Boyle's law set

(d) Chemistry: classification **(e)** Chemistry: bond order, note **(f)** Chemistry: ionic bond, image

(g) Biology: reaction, image **(h)** Biology: labelled image **(i)** Biology: ten-step process

Figure 4.1: Slides generated live from recorded lectures in physics, mathematics, chemistry and biology

4.6 Live Display and Teacher Control View

The FastAPI server sends every change as a versioned slide patch over WebSocket to three views. /display is the projector view and shows only the live slide. /view is opened by students through a shared link or QR code. /control (Fig. 4.2) is the teacher's view: an editable preview of the projected slide, navigation, pause, blank screen, new slide, add point, equation and image tools, the running transcript, mistake cards with both versions of a doubtful statement, the teacher's own PDF notes and the lecture structure tree (topic → facet → slides). The views never change the lecture state directly; they send commands that the core validates and applies.

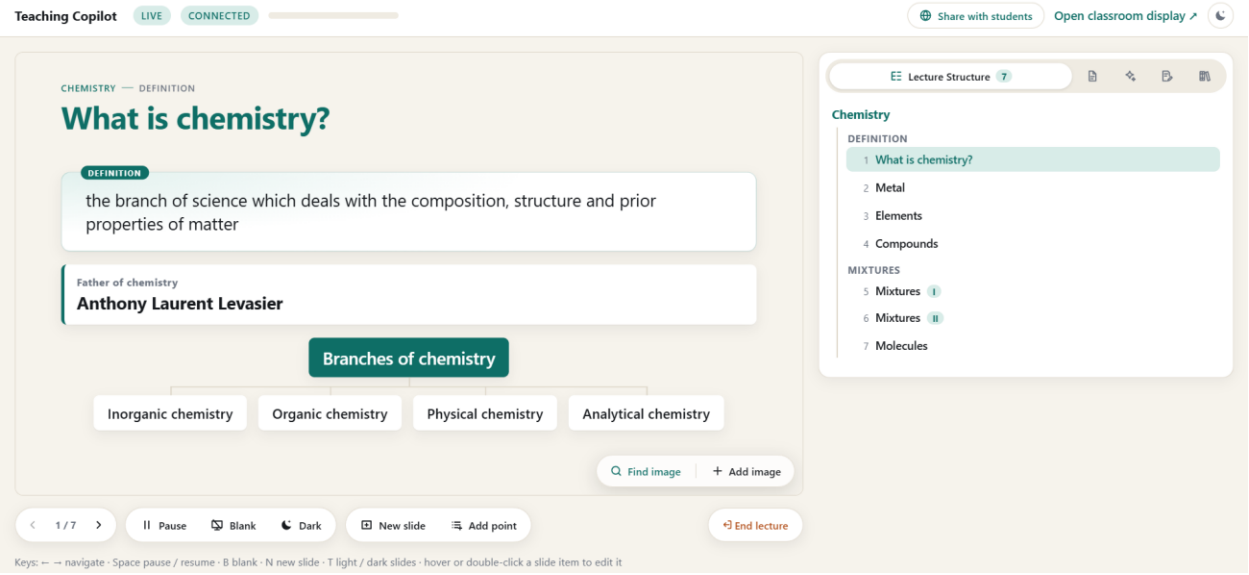


Figure 4.2: Teacher control view (/control) during a live chemistry lecture with the lecture structure tree

4.7 Concept Simulation (Version 2)

Concept Simulation adds live parametric simulations to the same deck. Each template is one Python module (slots with units and ranges, a parameter model, a solver, the elements to draw, setup and guard phrases) and one JavaScript scene that draws it as SVG on a single animation clock. The contract between them is a Pydantic model, so every simulation sent to the views is validated. Table 4.2 lists the 21 templates.

Table 4.2: Simulation templates implemented

Area	Templates	Computed outputs (examples)
Mechanics	Block on a surface, block on an incline, simple pendulum, projectile	Acceleration, net force, friction (static / kinetic), period, range, height
Electricity and waves	DC circuit (series / parallel), transverse wave, orbit, unit circle → sine	Equivalent resistance, currents, wave speed, orbital speed and period
Processes	Water cycle, human heart, photosynthesis, Bohr atom, acid–base neutralisation	Steps moved by speech, cardiac output, photon energy, pH and equivalence volume
Electronics	RLC step response, RLC AC (resonance, phasors), p-n junction	Damping ratio, f_0, impedance, phase, built-in potential, depletion width
Computer networks	Shortest-path routing (Dijkstra), TCP (handshake, window, loss)	Path and cost step by step, segments sent and resent
Optics and heat	Refraction (Snell's law), thin lens, ideal gas under a piston	Angle of refraction, critical angle, image distance, magnification, P V = n R T

Detection. The detector looks for a spoken cue together with a template's setup; context guards reject traps such as "orbital" in chemistry, "block diagram", "block of code", "slope of the line" or "integrated circuit". The policy decides whether to show, suggest (a chip in /control), update, switch, reopen or ignore. With the setting *Suggest* the teacher confirms each simulation.

Values. The parser reads spoken numbers ("five kilograms", "zero point two", "two hundred"), assigns

them to slots by unit, by the slot named nearby or by the slot changed last, understands relative phrases ("double the mass") and refuses what it cannot read safely ("minus 30 degrees"). A force counts only near a push word and not as a result ("the friction force is 9.8 newtons" is not a push). A value outside the slot's range is shown to the teacher as heard but not used.

Truthfulness. A value the template needs but nobody said (a default) moves the picture but is never printed. Computed values are marked *computed*. The teacher can set values in a Values panel; a value set by the teacher is final and speech never overwrites it.

Simulation mode. A shown simulation is a slide of its own (Fig. 4.3). While it runs, the engine makes no new slides; the interpretations are still collected. The simulation ends on "let's get back to the slides", the Back to slides button, a new topic or 180 s of silence about it, and a deterministic recap slide then shows the key points, rules and formulas said meanwhile.

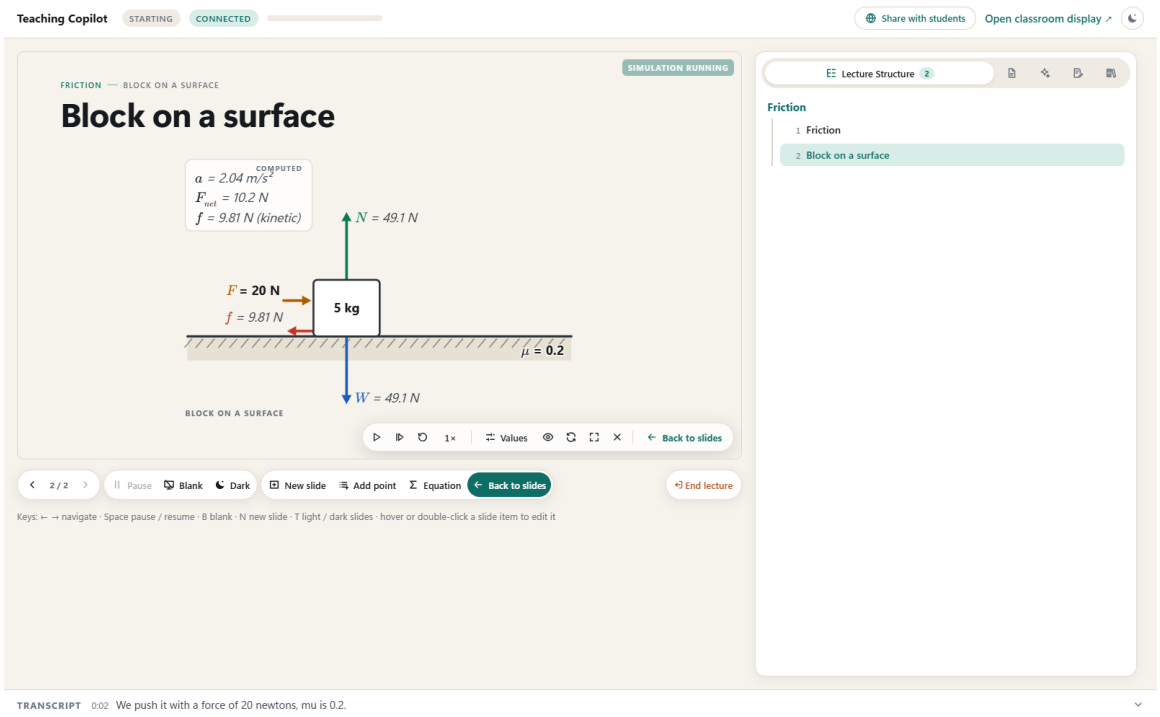
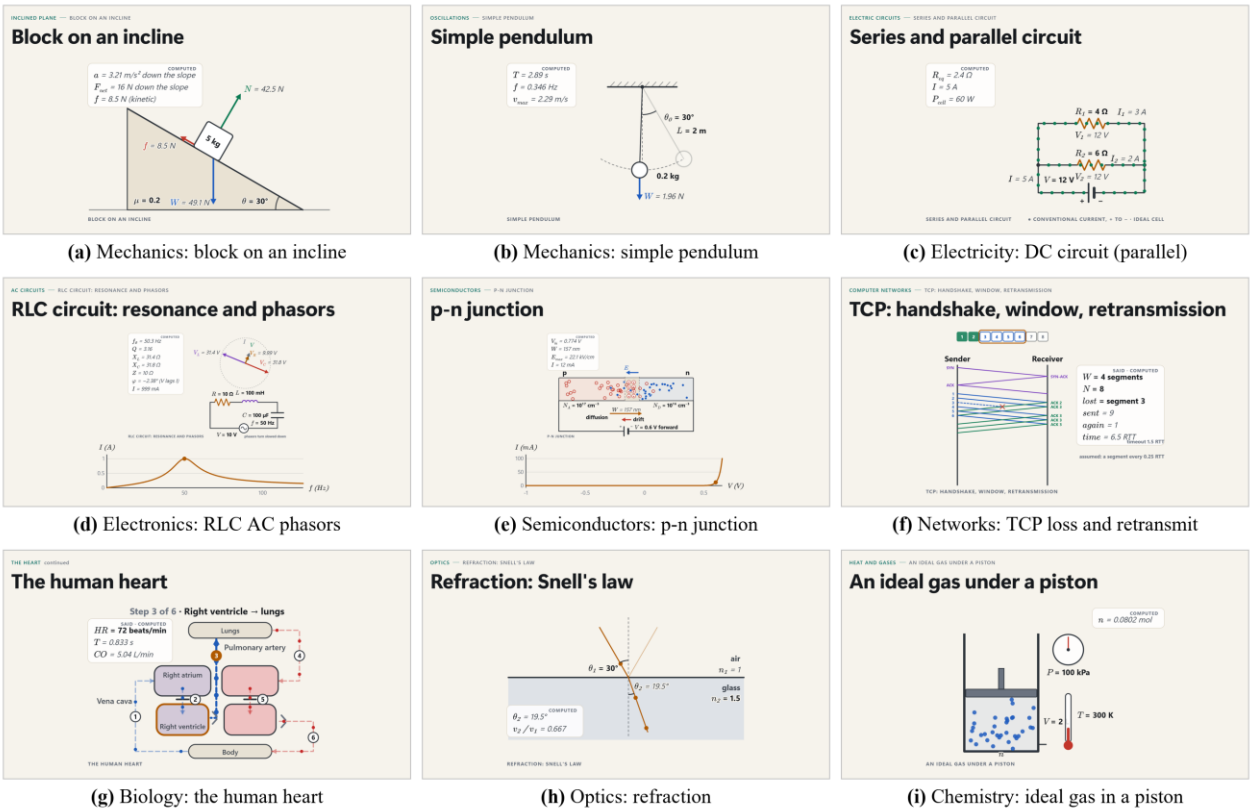


Figure 4.3: Simulation mode in /control: block on a surface with the computed card, the simulation bar (play, step, restart, speed, Values, Show, Full stage) and Back to slides



(a) Mechanics: block on an incline **(b)** Mechanics: simple pendulum **(c)** Electricity: DC circuit (parallel)

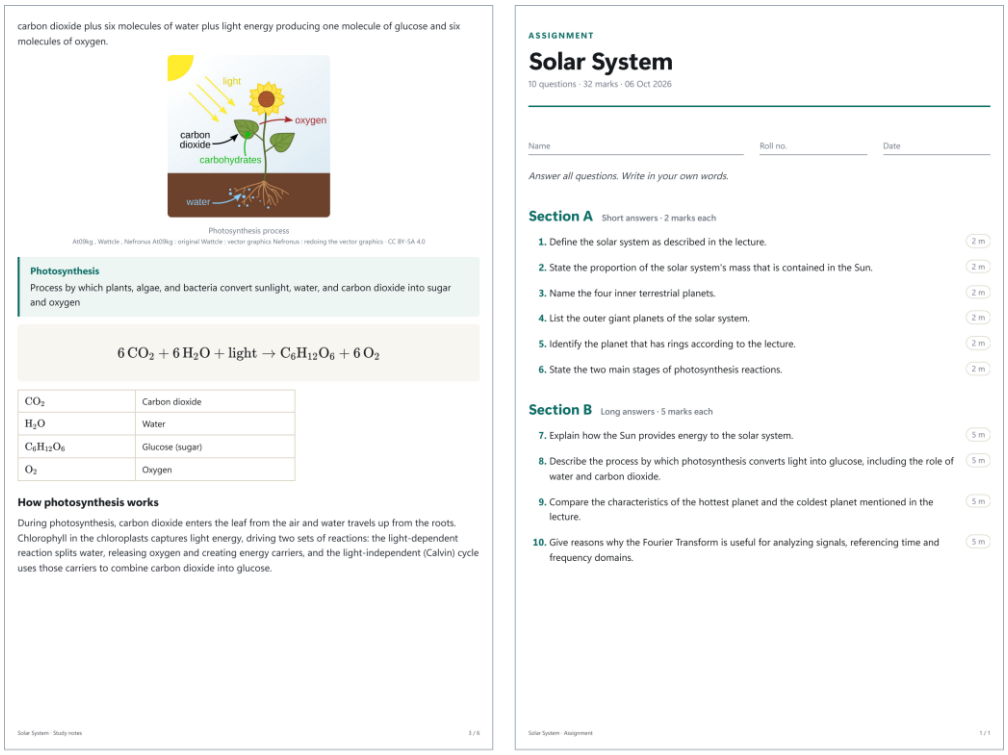
(d) Electronics: RLC AC phasors **(e)** Semiconductors: p-n junction **(f)** Networks: TCP loss and retransmit

(g) Biology: the human heart **(h)** Optics: refraction **(i)** Chemistry: ideal gas in a piston

Figure 4.4: Simulations from different subjects rendered by the implemented templates

4.8 Lecture Materials and Persistence

Every event of a lecture is written to an SQLite log. After a crash the lecture is restored from the log. After the lecture, the log produces study notes and an assignment as PDF (Fig. 4.5), a PPTX deck in which each simulation is drawn at its still moment, and a live quiz or test that students answer through the shared link. All materials are built only from the taught content and are edited by the teacher before use.



(a) Study notes (b) Assignment

Figure 4.5: Materials generated from the event log of a recorded lecture: (a) study notes page, (b) assignment

4.9 Testing Strategy

The project follows the rule that a feature is done only when it runs and produces the expected result. Testing was carried out at four levels:

- **Unit tests (1,547 passing):** filter, tracker, gate, guards, planner, composer, equation parser, value parser, detector and each simulation solver against hand-computed textbook values.
- **Browser tests (122 passing):** real Microsoft Edge driven by Playwright checks the display, control and student views, commands, both themes and simulations.
- **Replay tools (0 LLM tokens):** recorded lectures are replayed through the planner and the simulation detector to measure changes without spending quota.
- **Live runs:** recorded and simulated lectures with the real LLM, with screenshots of every layout inspected by the developers.

CHAPTER 5: EXPERIMENTATION AND RESULT ANALYSIS

5.1 Experimental Setup

All experiments were run on one laptop with an NVIDIA RTX 3050 GPU (4 GB video memory) and 16 GB of system memory. Two data sets were used (Table 5.1). D1, the development set, contains six lectures spoken by the authors (114.1 min); all thresholds were set by hand on D1. The held-out set D2 contains 30 lectures spoken live and unscripted by six speakers into the laptop microphone (Recorded; physics to economics) and six 10-minute public lectures played in as audio in real time (Public): MIT OpenCourseWare 7.016, 5.111, 18.01 and 8.01 and NPTEL Computer Networks and History of Economic Theory. The speakers consented to the recording. Teachers did not edit slide text during a test.

Table 5.1: Evaluation data

Set	Lectures	Minutes	Units	Fallback units	Slides
D1 (development)	6	114.1	389	45	163
D2 Recorded	30	349.2	1,275	59	453
D2 Public	6	60.0	344	13	52
D2 all (held out)	36	409.2	1,619	72	505

Units: groups of finished sentences sent for interpretation; fallback units were interpreted without the LLM.

Annotation and ratings. Two annotators independently annotated 18 lectures of D2 (12 Recorded, 6 Public; 191 min). Before seeing the slides, each marked topic boundaries and key facts in the transcript, then judged whether each key fact was on a slide and whether each slide statement was said and correct. Boundaries are scored by precision, recall and F1 within ±1 utterance and by Pk [23], the probability that two utterances half a segment apart are wrongly split or joined. Agreement is measured with Cohen's κ and Gwet's AC1. Five teachers (T1–T5) not involved in development rated 30 random D2 slides, each shown with what was said, from 1 to 5 on four criteria.

5.2 End-to-End Latency

Table 5.2 gives the timing of the live system from the session logs. Speech-to-text takes about 0.7 s per utterance and an LLM call about 2 s. Planning and composing take 2 ms. The median time from the end of a spoken line to its display is 11.2 s, of which 7.6 s is the gate's deliberate wait for a complete idea: the system is live, but not hard real-time. In two public lectures the free provider failed (timeout, HTTP 429); they are included, which raises the 95th percentile to 53.0 s. Without them the median is 10.2 s and the 95th percentile 23.4 s.

Table 5.2: Live system performance (seconds, from session logs)

Measure (s)	D1	D2 Rec.	D2 Pub.	D2 all
Speech-to-text per utterance	0.66	0.71	0.60	0.68
LLM call	1.35	1.92	2.23	1.96
Speech to slide, median	12.5	10.1	17.1	11.2
Speech to slide, 95th percentile	36.8	23.2	93.4	53.0

Medians unless stated; speech to slide: from the end of a spoken line to its display.

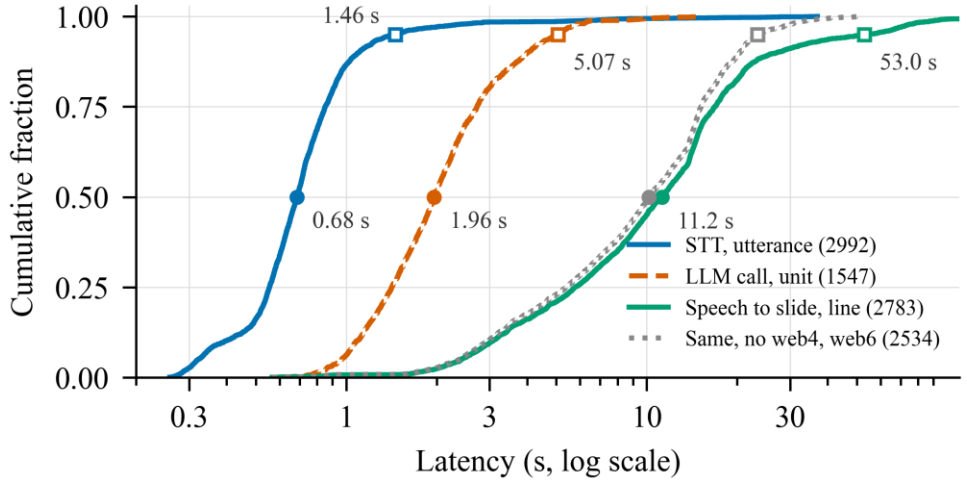


Figure 5.1: Cumulative distributions of speech-to-text time, LLM call time and end-to-end delay

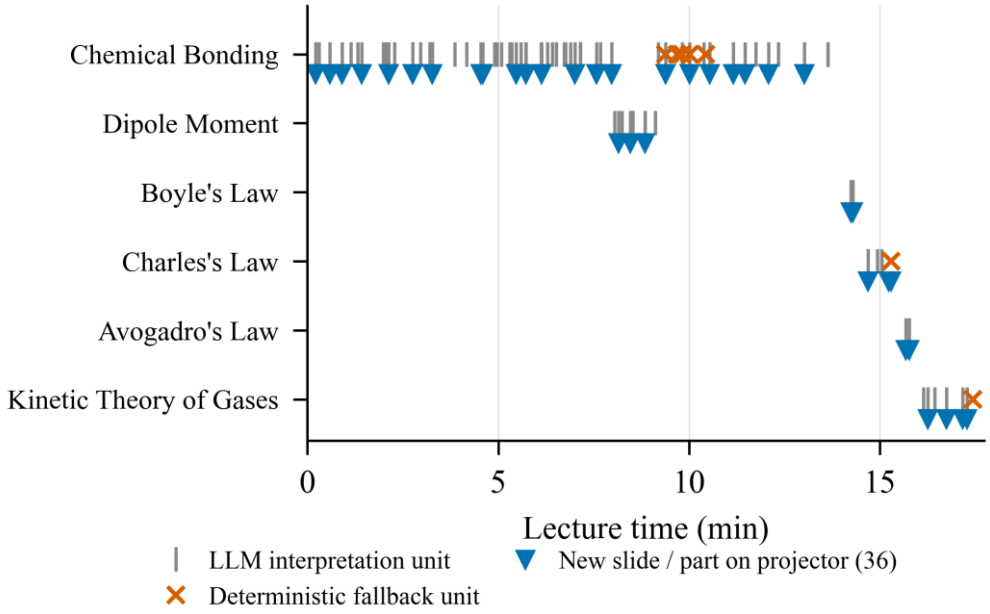


Figure 5.2: Timeline of a recorded chemistry lecture: LLM units, fallback units and new slides on the projector per topic

Fig. 5.2 shows a recorded chemistry lecture. Slides change in bursts while the teacher introduces new points, and short topics such as Boyle's law get their own slide quickly. Fallback units (crosses) occur when the free quota is briefly exhausted; the lecture continues without interruption.

5.3 Topic Boundaries and Content Accuracy

Table 5.3: Accuracy and teacher ratings (18 annotated D2 lectures)

Measure	Recorded	Public	All
Boundary precision ↑	0.81	0.73	0.78
Boundary recall ↑	0.54	0.19	0.42
Boundary F1 ↑	0.59	0.29	0.49
Pk ↓	0.29	0.37	0.32
Key facts on a slide ↑	88.0 %	84.0 %	86.6 %
Key facts without coverage guard	81.9 %	69.1 %	77.7 %
Statements not said ↓	0.9 %	0.9 %	0.9 %
Statements marked wrong ↓	3.6 %	4.0 %	3.7 %
Rating: correctness (1–5) ↑	4.20	4.06	4.15
Rating: relevance ↑	3.94	3.80	3.89
Rating: readability ↑	4.12	4.26	4.17
Rating: usefulness ↑	3.93	3.76	3.87
Boundaries agreement κ (raw)	0.81 (94 %)	0.56 (90 %)	0.72 (92 %)
Statement said agreement AC1 (raw)	0.89 (90 %)	0.84 (85 %)	0.88 (89 %)

↑ / ↓: *higher / lower is better. Mean of two annotators; boundaries within ±1 utterance.*

Observations. Slides change where the topic changes: boundary precision is 0.78 (95 % interval 0.68–0.88), so a projected slide is rarely replaced in the middle of a topic, and Pk is 0.32. Recall is moderate (0.42) by design: a new concept is not a new slide, so sub-points that annotators marked as boundaries are merged into the live slide, most often in the public lectures, which are dense lecture-hall talks. Of 669 key facts, 86.6 % reached a slide and only 0.9 % of 2,504 statement judgments were not supported by the speech. The coverage guard is essential: 60 covered facts reached a slide only through a sentence it restored; without it coverage would be 77.7 %.

Of the 3.7 % of statements marked wrong, 41 of 79 were also marked as said, i.e. a slip or mis-hearing of the speaker that the slide reproduced faithfully. Agreement between annotators was good for boundaries ($\kappa = 0.72$) and high for "supported" (AC1 0.88). The teachers rated the slides between 3.87 and 4.17 of 5, with readability and correctness highest.

Fallback-only lecture. One lecture ran entirely on the deterministic fallback because the quota was spent. It still covered 83.1 % of key facts with 0.7 % unsupported statements, but it rarely opened a new slide (recall 0.04) and 11 of its 13 slides were plain points. This shows that the fallback keeps the content and the LLM adds the structure.

5.4 Ablation of the Planner Rules

To find which rule produces the stability, the recorded signals and LLM answers of the 18 lectures were replayed through the planner with one rule changed at a time, without new LLM requests (Table 5.4). The unchanged replay gives the same decks as the live run.

Table 5.4: Planner ablation by replay (18 annotated D2 lectures)

Planner variant	Precision ↑	Recall ↑	Pk ↓	Changes / min	Replaced < 10 s ↓
Full planner (= live)	0.78	0.42	0.32	1.15	16
No dwell time	0.78	0.42	0.32	1.15	25
No topic confirmation	0.79	0.45	0.31	1.12	10
LLM labels decide	0.78	0.46	0.30	1.17	21
Each concept a slide	0.37	0.86	0.47	3.18	200

Replaced < 10 s: slides replaced within 10 s of being shown.

Analysis. The continuity rule matters most: if every concept gets its own slide, the deck changes 3.2 times per minute, 200 slides are replaced within 10 s of being shown and precision drops to 0.37. The dwell time does not change the boundaries but reduces quick replacements from 25 to 16. Topic confirmation made no measurable difference on D2 and is kept as a safeguard. When the LLM labels decide the boundaries but slides are still only extended, the scores hardly change, so the stability comes from the extend-only operations.

5.5 Comparison with an LLM-Controlled Baseline

An LLM-controlled baseline was run on the same 850 units of the 18 annotated lectures. For each unit it receives the slide it showed last and the last 60 s of transcript, and returns the slide to show, without the planner, guards or composer (same two models, about 0.5 million tokens).

Table 5.5: Proposed system vs. LLM-controlled baseline (same D2 units)

Measure	Proposed	LLM baseline
Seen statements rewritten / removed ↓	2.0 %	38.2 %
Seen statements lost from final deck ↓	6.2 %	65.8 %
Lectures with fewer rewrites ↑	18 of 18	0 of 18
Boundary precision ↑	0.78	0.72
Pk ↓	0.32	0.36
Content slides per minute	1.25	0.79

Rewritten: statements of a replaced slide version that the next version changed or removed.

Analysis. The baseline placed slide changes about as well (precision 0.72 against 0.78, $p = 0.34$), but it rewrote or removed 1,439 of the 3,763 statements students had already seen (38.2 %), against 91 of 4,515 (2.0 %) for the proposed system, and 65.8 % of what it showed was missing from its final deck. The difference holds in all 18 lectures; the mean per-lecture gap is 39 points (95 % interval 32–47; exact paired sign-flip permutation test, $p < 0.001$). Of the proposed system's 91 changes, 24 extended a statement, 6 followed a correction accepted by the teacher and 61 were composer merges of an elaboration.

5.6 Slide Output for Different Input Conditions

Table 5.6 shows how the composer chooses a representation from the kind of content spoken,

independent of the subject. Examples of each are visible in Fig. 1.1 and Fig. 4.1.

Table 5.6: Spoken input and the resulting slide representation

Spoken input (example)	Interpretation	Slide output
"Gravitation is the natural force of attraction between two objects with mass."	Definition	Definition card under the topic title
"F equals G m1 m2 by r squared."	Formula	Typeset formula with symbol legend
"First food is chewed, then saliva breaks down starch, then …"	Process	Numbered process flow (split into rows)
"Photosynthesis happens only in sunlight, respiration all the time …"	Comparison	Two-column comparison table
"In 1648 Van Helmont …, in 1771 Priestley …"	Timeline	Dated timeline
"Matter is classified into pure substances and mixtures …"	Classification	Classification tree
"The ionic bond forms between sodium and chlorine."	Concrete object	Text with a CLIP-checked open-licence image
"Discriminant is less than zero" (said as "discrement")	Mis-hearing	Shown as said; teacher card if suspected
"Okay, take out your notebooks."	Classroom talk	Filtered, no slide change

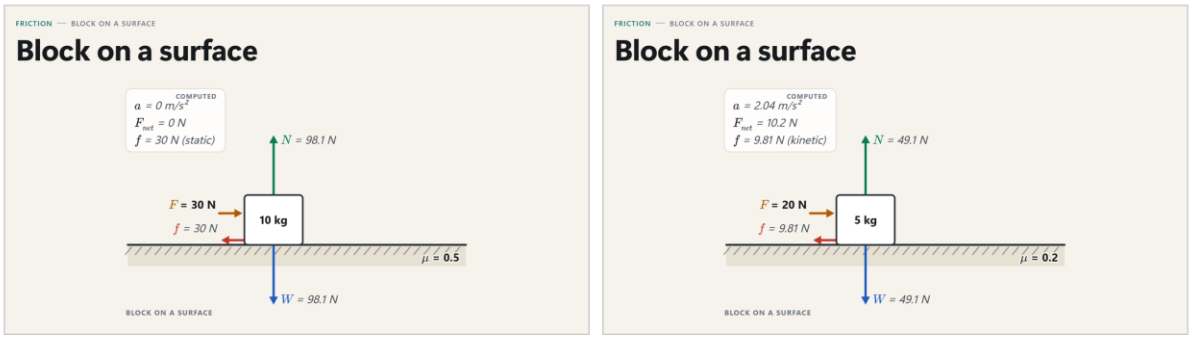
5.7 Concept Simulation: Outputs for Different Input Conditions

The simulation templates were run with different input values to check that the drawing and the computed values change as expected. All numbers in Tables 5.7–5.15 are the outputs of the implemented Python solvers (the same values printed on the projector), with g = 9.81 m/s².

5.7.1 Block on a Surface (friction)

Table 5.7: Block on a surface: push F = 20 N for different mass and friction

Mass (kg)	μ	Friction	Net force	Acceleration	Observation
5	0	—	20 N	4 m/s²	Frictionless: a = F/m
5	0.2	9.81 N (kinetic)	10.2 N	2.04 m/s²	Block slides
5	0.3	14.7 N (kinetic)	5.29 N	1.06 m/s²	More friction, less a
5	0.5	20 N (static)	0 N	0 m/s²	Push below μmg: stays at rest
10	0.2	19.6 N (kinetic)	0.38 N	0.038 m/s²	Heavier block barely moves



(a) 10 kg, F = 30 N, μ = 0.5: block stays at rest **(b)** 5 kg, F = 20 N, μ = 0.2: block slides

Figure 5.3: Block on a surface for two input conditions

Analysis. The solver switches correctly between static and kinetic friction: when the push is smaller than the limiting friction μmg, the friction equals the push and the block stays at rest; otherwise kinetic friction μmg acts and a = (F − μmg)/m.

5.7.2 Block on an Incline

Table 5.8: Block on an incline: m = 5 kg for different angles and friction

Angle θ	μ	Friction	Net force	Acceleration	Observation
10°	0.2	8.52 N (static)	0 N	0	tan θ < μ: block stays
30°	0.2	8.5 N (kinetic)	16 N down	3.21 m/s² down	Slides down
45°	0.2	6.94 N (kinetic)	27.7 N down	5.55 m/s² down	Steeper, faster
60°	0.2	4.91 N (kinetic)	37.6 N down	7.51 m/s² down	Normal force falls
30°	0	—	24.5 N down	4.91 m/s² down	a = g sin θ

5.7.3 Simple Pendulum and Projectile

Table 5.9: Simple pendulum (amplitude 10°) for different lengths

Length L (m)	Period T	Frequency	Speed at the bottom	Observation
0.5	1.42 s	0.704 Hz	0.386 m/s	T ∝ √L
1	2.01 s	0.498 Hz	0.546 m/s	≈ 2 s pendulum
2	2.84 s	0.352 Hz	0.772 m/s	√2 × the 1 m period
4	4.02 s	0.249 Hz	1.09 m/s	4× length, 2× period

Table 5.10: Projectile launched from the ground for different speeds and angles

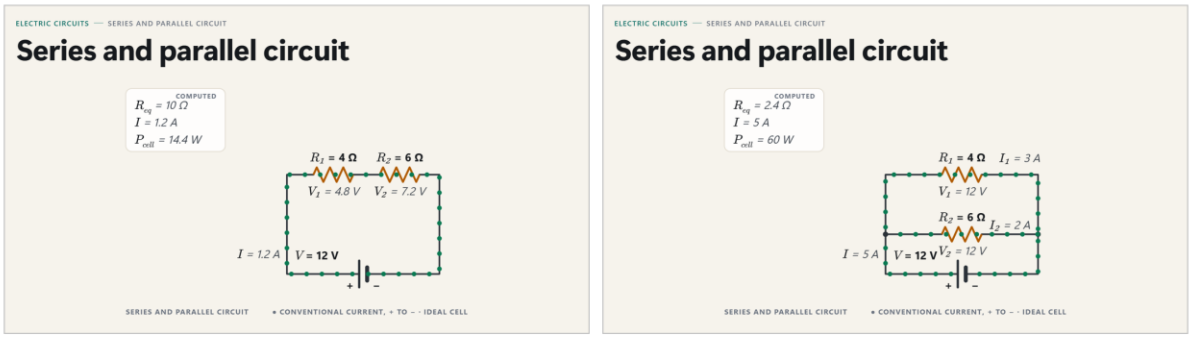
Speed (m/s)	Angle	Time of flight	Range	Max. height	Observation
20	30°	2.04 s	35.3 m	5.1 m	Same range as 60°
20	45°	2.88 s	40.8 m	10.2 m	Maximum range
20	60°	3.53 s	35.3 m	15.3 m	Higher, same range
30	45°	4.32 s	91.7 m	22.9 m	Range ∝ v²

5.7.4 Electric Circuits

Table 5.11: DC circuit and series RLC step response for different inputs

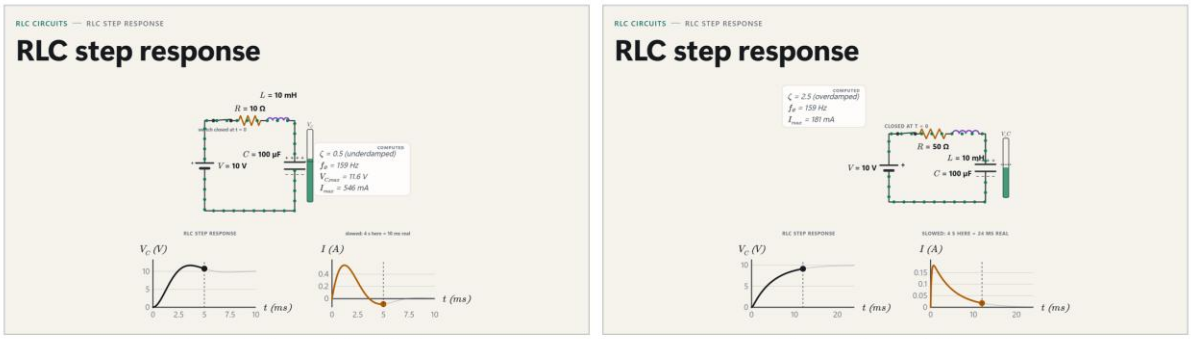
Input condition	Computed output	Observation
Series arrangement	R_{eq} = 10 Ω, I = 1.2 A, P = 14.4 W	Resistances add
Parallel arrangement	R_{eq} = 2.4 Ω, I = 5 A, P = 60 W	R_{eq} smaller than either
RLC, R = 20 Ω	ζ = 0.1 (underdamped), f_0 = 159 Hz, $V_{C,max}$ = 17.3 V, I_{max} = 86.3 mA	Overshoot and ringing
RLC, R = 200 Ω	ζ = 1 (critically damped), I_{max} = 36.8 mA	Fastest without overshoot
RLC, R = 1000 Ω	ζ = 5 (overdamped), I_{max} = 9.64 mA	Slow rise, no overshoot

DC circuit: V = 12 V, R_1 = 4 Ω, R_2 = 6 Ω. RLC: V = 10 V, L = 0.1 H, C = 10 µF.



(a) Series arrangement **(b)** Parallel arrangement

Figure 5.4: DC circuit: series and parallel arrangement of the same resistors



(a) Underdamped (small R) **(b)** Overdamped (large R)

Figure 5.5: RLC step response: underdamped and overdamped cases with V_C(t) and I(t) graphs

Table 5.12: Series RLC with AC supply (V = 10 V, R = 100 Ω, L = 0.1 H, C = 10 µF) at different frequencies

Frequency	X_L	X_C	Impedance Z	Phase	Current (rms)
50 Hz	31.4 Ω	318 Ω	304 Ω	−70.8° (V lags I)	32.9 mA
159 Hz (≈ f_0)	99.9 Ω	100 Ω	100 Ω	≈ 0°	100 mA
500 Hz	314 Ω	31.8 Ω	300 Ω	70.5° (V leads I)	33.4 mA

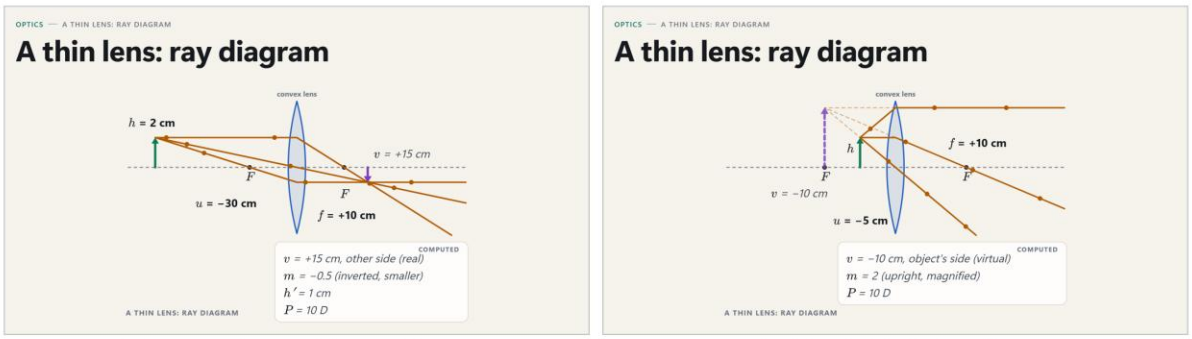
At resonance $X_L = X_C$, so Z = R and the current is maximum.

5.7.5 Optics

Table 5.13: Thin lens (f = 15 cm) for different object distances, Cartesian sign convention

Lens	Object distance	Image distance v	Magnification m	Nature of image
Convex	10 cm (inside f)	−30 cm (object's side)	+3	Virtual, upright, magnified
Convex	20 cm	+60 cm (other side)	−3	Real, inverted, magnified
Convex	30 cm (= 2f)	+30 cm	−1	Real, inverted, same size
Convex	45 cm	+22.5 cm	−0.5	Real, inverted, smaller
Concave	30 cm	−10 cm	+0.333	Virtual, upright, smaller

Power of the lens: +6.67 D (convex), −6.67 D (concave).

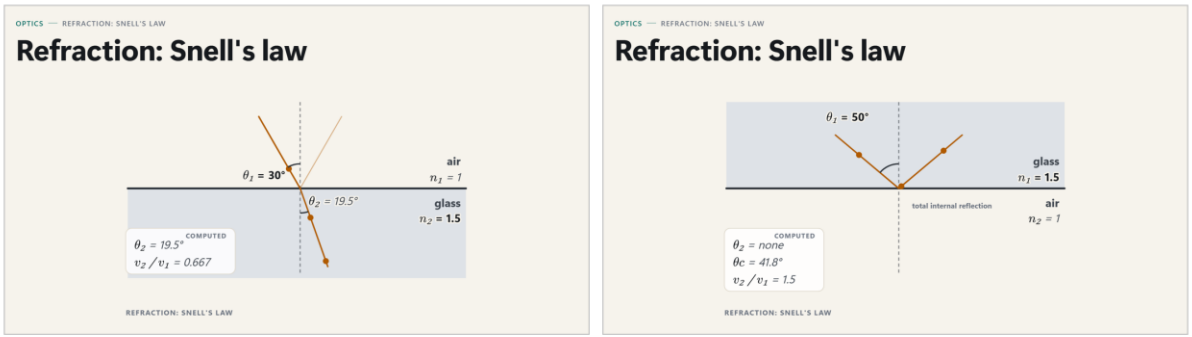


(a) Convex lens: real, inverted image **(b)** Object inside f: virtual image

Figure 5.6: Thin lens: real image and virtual image with the three principal rays

Table 5.14: Refraction at a plane boundary (Snell's law)

$n_1 \rightarrow n_2$	Angle of incidence	Angle of refraction	Critical angle	Observation
1.0 → 1.5 (air → glass)	30°	19.5°	—	Bends toward the normal
1.0 → 1.33 (air → water)	45°	32.1°	—	$v_2 / v_1 = 0.752$
1.5 → 1.0 (glass → air)	30°	48.6°	41.8°	Bends away from the normal
1.5 → 1.0 (glass → air)	50°	none	41.8°	Total internal reflection



(a) Air to glass: ray bends toward normal **(b)** Glass to air beyond critical angle: TIR

Figure 5.7: Refraction: air to glass and total internal reflection from glass to air

5.7.6 Semiconductors, Gases, Waves and Orbits

Table 5.15: Further templates with different inputs

Template	Input condition	Computed output
p-n junction (Si, $N_A = N_D = 10^{16}$ cm^{-3})	Unbiased (0 V)	$V_{bi} = 0.714$ V, W = 430 nm, $E_{max} = 33.2$ kV/cm
	Forward 0.5 V	W = 235 nm, $E_{max} = 18.2$ kV/cm
	Reverse 5 V	W = 1.22 µm, $E_{max} = 94$ kV/cm
Ideal gas, V = 2 L, T = 300 K	P = 100 kPa	n = 0.0802 mol
	P = 200 kPa	n = 0.16 mol
Transverse wave	λ = 2 m, f = 5 Hz	v = 10 m/s, T = 0.2 s
Orbit around the Earth	Altitude 400 km	v = 7.67 km/s, T = 92.4 min
	Altitude 35,786 km	v = 3.07 km/s, T = 23.9 h (geostationary)
	Radius 384,400 km (Moon)	v = 1.02 km/s, T = 27.5 days



(a) Forward bias: narrower depletion region

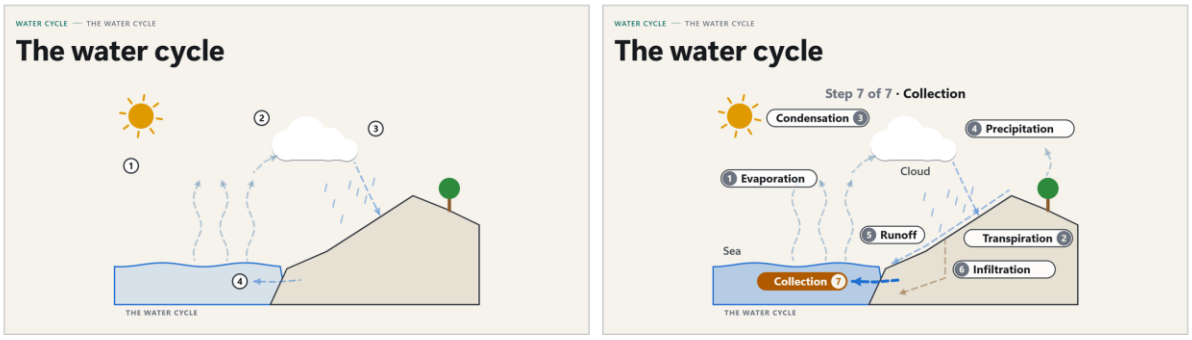
(b) Reverse bias: wider depletion region

Figure 5.8: p-n junction under forward and reverse bias

Analysis. In every case the outputs agree with the textbook formulas: the depletion width grows with reverse bias and shrinks with forward bias, the gas amount doubles with the pressure at fixed volume and temperature, and the computed orbit at 35,786 km gives the one-day geostationary period. Unit tests compare each solver with hand-computed values to within 0.1 %.

5.7.7 Processes and Discrete Events

Process templates move step by step as the teacher names each step; a step is never named on the screen before it is said. Routing shows Dijkstra's algorithm one visited node at a time and prints the path and its cost only at the final step; it never runs ahead of the teacher (Figs. 5.9 and 5.10).



(a) Process at its first step **(b)** All steps reached by speech

Figure 5.9: Water cycle: first step and all steps reached by speech



(a) Dijkstra: one step as spoken **(b)** Shortest path and its cost

Figure 5.10: Shortest-path routing (Dijkstra): an intermediate step and the final path

5.8 Simulation Detection and Value Parsing

Table 5.16: Spoken sentences and the simulation system's response

Spoken sentence	Detected template	Values read
"Consider a block of mass five kilograms on a rough floor."	Block on a surface	m = 5 kg; friction exists (μ not printed)
"The coefficient of friction is zero point two."	(open simulation)	μ = 0.2
"Now we push it with 20 newtons."	(open simulation)	F = 20 N
"Now imagine the same block on an inclined plane at 30 degrees."	Switch to incline	θ = 30°; mass and μ carried over
"Make the angle 95 degrees."	(open simulation)	Heard 95° – not used (range 0–89°)
"An object is placed 30 centimetres from a convex lens of focal length 15 centimetres."	Thin lens	u = 30 cm, f = 15 cm, convex
"Two resistors of 4 ohm and 6 ohm are connected in parallel."	(DC circuit open)	R_1 = 4 Ω, R_2 = 6 Ω, parallel
"The friction force is 9.8 newtons."	(open simulation)	Nothing (a result, not a push)

The detector was evaluated with a 0-token replay over 66 recorded and scripted lectures, including all recorded lectures without any simulation moment. It showed a simulation with precision 1.00 (16 of 16) and recall 1.00, raised no simulation or suggestion on the negative lectures and raised 9 of 10 annotated suggestion moments. The value parser reached precision and recall 1.00 per slot on a corpus of 82 spoken lines. A simulated RLC lecture with the real LLM confirmed the end-to-end path (cue →

step response with V, R, L, C read → AC simulation → end cue → recap slide) and found two defects that were fixed afterwards.

5.9 Discussion of Results

- The extend-only planner is the main reason for stability: 2.0 % of seen statements changed against 38.2 % for an LLM-controlled design, without loss in boundary precision.
- Transcript grounding works: 86.6 % of key facts reached a slide and only 0.9 % of statements were not said; the coverage guard alone contributes 9 percentage points.
- The system remains usable when the LLM fails, because the deterministic fallback keeps the content.
- Latency of about 11 s is acceptable for a live display that waits for a complete idea; it is dominated by the gate, not by computation.
- Simulations add 0 LLM tokens, appear within about one second of the spoken cue and print only said or computed values.

CHAPTER 6: CONCLUSION

6.1 Advantages

- Turns the teacher's live explanation, including unplanned examples, into structured slides during the lecture.
- Stable: slides are only extended, so students can copy without the text changing under them.
- Faithful: content is checked against the transcript; suspected mistakes go to the teacher, not the class.
- Teacher in control: edit, pause, blank, add images or equations, and override every decision.
- Live simulations with correct, traceable numbers for science and engineering topics.
- Low cost: one laptop with a 4 GB GPU, local speech and vision models, free LLM tier, no paid API.
- Robust: continues with a deterministic fallback when the network or LLM fails; recovers after a crash.
- Accessible: a structured text and visual record helps students with hearing difficulties and slow note-takers.
- Reusable: the event log produces notes, assignments, quizzes and PPTX decks after the lecture.

6.2 Limitations

- English speech only; accuracy depends on microphone quality and the speaker's accent.
- Median delay of about 11 s; not hard real-time, and free-tier outages raise the delay.
- Boundary recall is moderate (0.42) by design, because sub-points are merged into the live slide.
- The slides follow the speech: a slip of the teacher is reproduced unless the teacher accepts a correction.
- Simulations exist only for the 21 developer-written templates; a new topic needs a new template.
- The annotators were the authors (annotation was done before seeing the slides); no classroom study of learning outcomes has been carried out yet.

6.3 Applications

- Live classroom lectures in schools, colleges and universities.
- Online and hybrid classes, where remote students follow the /view link.
- Inclusive education for students with hearing impairment or learning difficulties.
- Training sessions, workshops and seminars in industry.
- Quick creation of study material, assignments and quizzes from an actual lecture.

- Laboratory and tutorial sessions that need quick simulations of physical and technical systems.

6.4 Conclusion

The project designed, implemented and evaluated a Multimodal AI Teaching Copilot that converts a teacher's live speech into a structured, continuity-aware slide deck and live concept simulations on the classroom projector. The key design decision was to keep the LLM as a bounded, schema-validated interpreter inside a deterministic core: the model labels small units of speech, while a deterministic planner decides the slide sequence and only extends the deck. On 36 held-out lectures run on a 4 GB GPU laptop, the median delay from speech to slide was 11.2 s; on 18 annotated lectures, slide changes matched topic boundaries with precision 0.78, 86.6 % of key facts reached a slide, only 0.9 % of statements were unsupported, and teachers rated the slides 3.87–4.17 of 5. Compared with an LLM-controlled baseline, the design reduced rewritten statements from 38.2 % to 2.0 % in every lecture. The Concept Simulation version adds 21 parametric templates whose numbers are only said, set or computed, with perfect show precision and recall on the evaluated lectures. The objectives stated in Chapter 3 have been met, and the results show that useful, stable and faithful visual teaching material can be produced live from ordinary speech on low-cost hardware.

6.5 Future Scope

- **Explore mode:** students change simulation values on their own devices and see the output change, while the projector stays unchanged.
- **Story Scenes (V3):** narrated illustrated scenes for history, literature and biology topics using the same core.
- **Multilingual output (V4):** Hindi, Marathi and other Indian languages for speech and slides.
- More simulation templates and a faster way for teachers to add templates of their own.
- Local or paid low-latency LLMs to reduce the delay below 5 s and remove free-tier outages.
- A classroom study of learning outcomes with control groups.

REFERENCES

[1] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[2] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Sci.*, vol. 12, no. 2, pp. 257–285, 1988.

[3] R. E. Mayer and R. Moreno, "Nine ways to reduce cognitive load in multimedia learning," *Educ. Psychologist*, vol. 38, no. 1, pp. 43–52, 2003.

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023, pp. 28492–28518.

[5] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. NeurIPS*, vol. 33, 2020, pp. 5776–5788.

[6] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, Art. no. 102274, 2023.

[7] B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv:2307.09702, 2023.

[8] R. S. Kushalnagar, W. S. Lasecki, and J. P. Bigham, "Accessibility evaluation of classroom captions," *ACM Trans. Accessible Comput.*, vol. 5, no. 3, Art. no. 7, 2014.

[9] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[10] T.-J. Fu, W. Y. Wang, D. McDuff, and Y. Song, "DOC2PPT: Automatic presentation slides generation from scientific documents," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 1, 2022, pp. 634–642.

[11] E. Sun, Y. Hou, D. Wang, Y. Zhang, and N. X. R. Wang, "D2S: Document-to-slide generation via query-based text summarization," in *Proc. NAACL-HLT*, 2021, pp. 1405–1418.

[12] E. Xie et al., "Leveraging grounded large language models to automate educational presentation generation," in *Proc. Large Found. Models Educ. Assessment*, PMLR, vol. 264, 2025, pp. 207–220.

[13] H. Zheng et al., "PPTAgent: Generating and evaluating presentations beyond text-to-slides," in *Proc. EMNLP*, 2025, pp. 14402–14418.

[14] J. Ge et al., "AutoPresent: Designing structured visuals from scratch," in *Proc. IEEE/CVF CVPR*, 2025, pp. 2902–2911.

[15] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021, pp. 8748–8763.

[16] H. Gonzalez et al., "Automatically generated summaries of video lectures may enhance students' learning experience," in *Proc. 18th Workshop Innovative Use NLP Build. Educ. Appl. (BEA)*, 2023, pp. 382–393.

[17] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *Int. J. Artif. Intell. Educ.*, vol. 30, no. 1, pp. 121–204, 2020.

[18] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, 2025.

[19] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.

[20] M. A. Hearst, "TextTiling: Segmenting text into multi-paragraph subtopic passages," *Comput. Linguistics*, vol. 23, no. 1, pp. 33–64, 1997.

[21] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[22] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. ACL Workshop Text Summarization Branches Out*, 2004, pp. 74–81.

[23] D. Beeferman, A. Berger, and J. Lafferty, "Statistical models for text segmentation," *Mach. Learn.*, vol. 34, no. 1–3, pp. 177–210, 1999.

[24] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," GitHub. [Online]. Available: https://github.com/SYSTRAN/faster-whisper

[25] Silero Team, "Silero VAD: Pre-trained enterprise-grade voice activity detector," GitHub, 2021. [Online]. Available: https://github.com/snakers4/silero-vad

APPENDIX A: DETAILS OF PAPER PUBLICATION

Item	Details
Title of paper	Multimodal AI Teaching Copilot: Live Slide Generation from Lecture Speech
Authors	Prachi Potdar, Paritosh Raut, Deepali Magdum, Swati Kale, Sunita Patnaik
Affiliation	Department of Electronics and Telecommunication Engineering, JSPM's Rajarshi Shahu College of Engineering, Pune, Maharashtra, India – 411033
Format	IEEE conference format, 6 pages, 31 references
Name of the conference / journal	[To be filled]
Paper ID / submission date / status	[To be filled]
Comments of reviewers	[To be attached]
Certificate	[To be attached]

Abstract of the paper. Prepared slides cannot show what a teacher adds while explaining, and a large language model (LLM) that regenerates slides from a growing transcript rewrites what students are reading. The paper presents a teaching copilot that builds a structured, continuity-aware slide deck on the classroom projector from the teacher's live speech. Locally transcribed speech is sent in bounded units to an LLM that only returns a schema-validated interpretation; a deterministic planner turns it into slide operations that only extend the deck, and a transcript-coverage guard restores facts the LLM omits. Across 36 held-out lectures the median delay from speech to slide was 11.2 s; on 18 annotated lectures boundary precision was 0.78, 86.6 % of key facts reached a slide and 0.9 % of statements were unsupported; an LLM-controlled baseline rewrote 38.2 % of seen statements against 2.0 %.

APPENDIX B: IMPLEMENTATION OF PAPER, COPYRIGHT CERTIFICATE, PLAGIARISM REPORT

B.1 Implementation of the paper. The method described in the paper is implemented in full in this project (Chapters 3 and 4). The paper's evaluation (Chapter 5, Sections 5.1–5.5) was produced with the project's own tools from the recorded event logs: end-to-end latency from live logs, annotation workbooks and rating forms, the zero-token planner ablation by replay and the LLM-controlled baseline.

B.2 Copyright certificate. [To be attached]

B.3 Plagiarism report. [To be attached]

SDG MAPPING OF PROJECT

The project contributes to the following United Nations Sustainable Development Goals (SDGs):

Sustainable Development Goal	Target	Contribution of the project	Level
SDG 4: Quality Education	4.1, 4.3	Live structured slides, simulations and study material improve the quality of classroom teaching in schools and higher education.	High
SDG 4: Quality Education	4.5, 4.a	Structured live text and visuals support students with hearing impairment and slow note-takers; works with low-cost hardware in existing classrooms.	High
SDG 4: Quality Education	4.c	Reduces teachers' preparation load and gives them material (notes, assignments, quizzes) from their own lectures.	Medium
SDG 9: Industry, Innovation and Infrastructure	9.5, 9.c	Applied research in AI for education; runs on a single laptop with free and local components, increasing access to ICT.	Medium
SDG 10: Reduced Inequalities	10.2	Free and low-cost design makes AI-assisted teaching available to institutions that cannot afford commercial tools.	Medium